

TyreMind — The Complete Data & Progress Document

Audience: the TyreMind team and our mentors.

Purpose: everything we hold, everything we tested, everything we found, what we got wrong, and exactly what we build next. Written so it can be **read aloud** — if you get stuck, read the line off the page and it will sound right.

Status: internal. Not committed and not pushed. It contains our honest self-assessment, including six failed hypotheses and six retracted claims — which is exactly what makes it useful for preparing for questions.

Contents

Part	What's in it	Read it when
0	What to say in the meeting — in simple words. Script, word list, questions and answers	Read this one. Skip the rest unless asked.
1	Every dataset we hold, in full detail	They ask "what data do you have?"
2	Which real sensors produce each field at a real Grand Prix	They ask "where does this come from?"
3	What does not exist anywhere on the internet	They ask "why not just measure the tyre?"
4	What each dataset was used for	Quick reference table
5	All 32 experiments — question, method, result	The core of the work
6	Every claim, graded real / caveated / refuted	They ask "how sure are you?"
7	The 326 automated tests	They ask "how do you know it still works?"
8	How to rebuild anything	For the team
9	The 14 experiments we run next	They ask "what's next?"
10	How we build on the existing TrackShift work	They ask "how is this different?"
11	Their four future opportunities, designed	They ask "where does this go?"
12	The plan, in order	The closing slide
12B	Step-by-step build plan for 4 people	When they ask "how will you actually do it?"
13	How to download and share this	For you

The thirty-second version

We are solving Problem 3: *pull the true tyre wear rate out of a practice session, after removing fuel weight, traffic and track evolution.*

- **203 real F1 sessions, 91,867 clean laps, 84 event-seasons, 2022–2025.** All public timing data. We downloaded it ourselves.
 - The **one thing nobody on Earth can download** is the answer key: measured tyre wear. Pirelli and the teams have it; the public does not. Part 3 is entirely about how we worked around that.
 - We ran **32 experiments. Thirteen of our own ideas failed** and we kept the negative results rather than hiding them.
 - Every headline number is reproducible from a script in this repo.
-

PART 0 — WHAT TO SAY IN THE MEETING

Simple words. Short lines. Read straight from this page.

1. The problem (say this first)

"A lap time hides three things at once.

The tyre gets old, so the car gets **slower**. The fuel burns off, so the car gets **lighter and faster**. The track gets more rubber, so everyone gets **faster**.

We only see the final lap time. Our job is to pull out only the tyre part."

Then add why it is hard:

"Fuel is worth about 0.08 seconds per lap. The tyre is worth about 0.03 to 0.08 seconds per lap. So the thing hiding the answer is as big as the answer itself.

If you don't remove fuel properly, you don't get a small mistake. You get the **opposite sign**. We checked 77 real races. The simple method says the tyre got **faster** as it got older in 74% of them. That is impossible."

2. Our five main points

#	Point	Number to say
1	We have a lot of real data	203 sessions, 91,867 laps, 4 seasons
2	The simple method is not a little wrong. It is broken.	74% of races give an impossible answer
3	When a true answer exists, we find it	Our error 0.0044, theirs 0.0966
4	Our error bars are honest	We say 95%, we are right 95.2% of the time
5	We know the limit of what is possible, and we are near it	1.38× the best possible

If you only get 5 minutes, say these 5 lines and stop.

3. Words they may ask about

Word	Say this
Degradation rate	Seconds lost per lap because the tyre is wearing out.
Stint	One run on one set of tyres, from fitting to pitting.
Tyre age	How many laps the tyre has done. Not how worn it is. Nobody publishes that.
Compound	HARD, MEDIUM, SOFT. These are only weekend names. Pirelli picks 3 of its 5 rubbers each weekend and renames them.
Confounder	Something that changes lap time but is not the tyre. Fuel, traffic, track.
State-space model	A model for something you cannot see. You guess the tyre condition from the lap times you can see.
Kalman filter	The normal tool to track something invisible using noisy readings. GPS and aeroplanes use it.
Prior	A physics assumption we add before looking at data. It guides the answer. It does not decide it.
Conformal prediction	A way to make an error bar that is really correct the number of times it claims.
Coverage	How often we were actually right. If we say 95% and we are right 75% of the time, we are lying.
Collinear	Two things that move together exactly, so you cannot tell them apart.
RUL	Remaining Useful Life. How many laps are left before the tyre drops off.
The cliff	The point late in a stint where the tyre suddenly gets much worse.
MAE	Average size of our mistake.
Bias	Average mistake with direction. Are we always too high or too low?
p-value	Small p = the pattern is probably real. We used 0.05.

4. Questions they will ask

Q. What data do you have?

"203 real F1 sessions. 91,867 clean laps. Four seasons, 2022 to 2025. It comes from FastF1, which reads F1's official timing feed. Each lap gives us the driver, lap time, tyre compound, tyre age, and lap number. Eight columns. That is all anyone has in public."

Q. Is the data clean?

"Not when it arrives. It has pit laps, safety car laps, and bad laps. We use seven filters. And we save a file for every session showing what we removed. For Monza 2024 we got 1,008 laps and kept 921. We can show exactly where the other 87 went."

Q. Why can't you just measure the tyre?

"Because nobody shares it. Pirelli checks every tyre after every session. That data goes to Pirelli, the FIA and the teams. It has never been made public. We searched everywhere. It does not exist. So we work it out from lap times only."

Q. What model did you use? Why not deep learning?

"A state-space model with a Kalman filter, written by us.

Three reasons. One, the tyre condition is invisible, and this is the standard tool for invisible things. Two, it gives honest uncertainty. Three, it explains itself — it says how much was fuel and how much was tyre.

We did test deep learning. The neural network came **last of six**. LightGBM predicted lap times fine, but its 95% error bar was only right 61% of the time."

Q. How do you know it works, if there is no true answer? *(the hard one — give all 3)*

"Three ways.

One. We made our own data where we know the true answer. We hide it, the model guesses, we compare. Our error was 0.0044. The simple method was 0.0966. We were right inside our error bar 25 times out of 25.

Two. Practice predicts the race. We guess from Friday practice, then check Sunday. 42 events. Our error 0.081, simple method 0.147. But I must be honest: on real data there is no true answer, so we compare against our own model's race result. So this shows we agree with ourselves. It is not proof. Fixing this is our next experiment.

Three. Some answers are simply impossible. A tyre cannot get faster as it wears. The simple method says it does, in 74% of races. You don't need the right answer to know that answer is wrong."

Q. Why are your error bars so wide?

"Because they are honest. Our model said 95% but was right only 75% of the time. We fixed it. Now we say 95% and we are right 94.7%. The honest bar is 2.3 times wider. We chose the wide one. A wrong pit call costs a race."

Q. Does it work live?

"Yes. It updates every lap. And the error bar **fixes itself while the race is running**. Over 69,206 laps it landed at 95.2%, against a 95% target. It also shows its own accuracy live, so you can check it as it runs."

Q. What is the maths result you are proud of?

"Inside one stint, tyre age goes up by 1 each lap. Fuel burned also goes up by 1 each lap. They move together **exactly**. So the maths can only find the *difference* between them, never the two separately. More data does not help. It is algebra.

We measured it. 520 stints, all 100% locked together, correlation 1.000.

There is one escape. A tyre curve **bends** — it warms up early and falls off late. Fuel is a straight line and never bends. That bend is the only clue, and it is only **6%** of the information. The other 94% must come from physics.

We also worked out the best precision anyone could ever get. We are at 1.38 times it. So we can tell you our answer *and* how much room is left above us."

Q. Why did you test on jet engines?

"To prove it is not just an F1 trick. We took our code, changed nothing, and ran it on NASA's jet engine data. Tyre stint becomes engine life. Tyre age becomes flight hours. We got 22.7 error against a normal range of 12 to 25. And 44% of our guesses were early, which is the safe side for maintenance."

Q. What failed? *(say this proudly)*

"Six of our own ideas failed and we kept all six in the report.

Energy instead of laps — no difference. Real Pirelli compound instead of the label — 6% worse. Track shape to predict a new circuit — no effect. Temperature explaining our error — explains nothing. Matching stint lengths — made it worse. Driver style — made predictions worse.

The fifth one is the best. We found a clue, built a test that could kill our own idea, and it did. That is real research, not a dashboard."

Q. How is this better than the last team's work?

"Their metrics are good. We are keeping them. We are changing what calculates them.

One. Their formula is tuned on **one circuit**, Bahrain. We tested 26 circuits and found that track-to-track transfer does not work. So one circuit will not travel.

Two. Their alerts use fixed thresholds on a single number. A threshold with no error bar is a coin toss near the line. We put a proper range underneath, and keep their 18 rules on top as a safety check. When the model and the rules disagree, the engineer sees both.

Three. They show only what worked. We show six things that failed."

Q. Can it scale? Is it cheap?

"Yes. It fits a session in 12 seconds on a laptop. The data is 2.6 MB for four seasons. No GPU. No cloud bill. It runs fully offline — you can clone it and use it on a plane. The neural network we tested took 36 seconds and came last."

Q. What is the business value?

"Three levels. A team: one wrong pit call costs positions, and positions cost millions. A broadcaster: it is a graphics and insight product. And the big one: the engine is not F1-only. We proved it on jet engines. The same code fits aircraft fleets, wind turbines, and factory machines."

Q. What is next?

"Three things. Fix the one weak test I mentioned. Check ourselves against a tyre model we did not build. And add puncture risk, using real failure records that we found are free and public."

5. Never say these

✗ Do not say	✓ Say instead
"The FastF1 community uses the broken method."	"The simple method — the first thing anyone tries."
"Our practice-to-race test proves we are right."	"It shows we agree with ourselves. It is not proof."
"Track shape is harmful."	"Track shape has no effect. Only temperature was harmful."
"We beat five other models."	"We are best at finding the wear rate. At plain lap-time guessing we are third."
"We measure tread in millimetres."	"We give percentage of life left. Depth cannot be measured from public data."
"Drivers differ in tyre care."	"There is a tiny effect between teammates, but it does not help predictions."

One rule: if they ask something we have not tested, say **"we have not tested that yet, it is on our list."** That costs nothing. A wrong guess costs everything.

PART 1 — EVERY DATASET WE HOLD

There are five separate bodies of data. They are listed in order of importance.

1.1 The season corpus — our main dataset

Where it lives: `data/season/` — 204 files, 2.6 MB total (Parquet, compressed). **What it is:** one file per F1 session, named like `2024-italian-grand-prix-R.parquet`.

Size:

Sessions	203 usable
Clean racing laps	91,867
Event-seasons covered	84 (e.g. "Monza 2024" is one event-season)
Seasons	2022, 2023, 2024, 2025

Split by season: 2022 → 15 sessions, 2023 → 57, 2024 → 67, 2025 → 64. (2022 is thin because FastF1's tyre-age data for that season is patchier; we kept only the sessions that passed our quality gate.)

Split by session type: Race → 84, FP1 → 44, FP2 → 49, FP3 → 26.

The columns (this is the whole schema — there are only eight)

Column	Type	Meaning in plain words
<code>driver</code>	text	Three-letter code, e.g. <code>VER</code> , <code>HAM</code> , <code>LEC</code> .
<code>session_lap</code>	integer	Which lap of the <i>session</i> this was. Lap 1, lap 2, ... Used to measure track evolution (the track gets faster as rubber goes down).
<code>run_id</code>	integer	A unique number for one continuous stint on one set of tyres. Every time a driver pits, a new <code>run_id</code> starts.
<code>tyre_age</code>	float	How many laps old the tyre set is. Note: this can start above zero, because a driver may fit a used ("scrubbed") set.
<code>lap_time</code>	float	The lap time in seconds, e.g. <code>84.853</code> . This is our only measured target variable.
<code>compound</code>	text	<code>SOFT</code> , <code>MEDIUM</code> or <code>HARD</code> for that weekend.
<code>traffic_index</code>	float 0–1	How badly the lap was compromised by the car in front. 0 = clear air, 1 = stuck right behind someone. We compute this ourselves (see 1.1.4).
<code>lap_in_run</code>	float	How many laps the driver has completed since this stint began . This is our fuel proxy — the car burns fuel every lap, so it gets lighter and faster.

An example of three real rows (2022 Australian GP, race):

driver	session_lap	run_id	tyre_age	lap_time	compound	traffic_index	lap_in_run
ALO	2	2	2.0	87.059	HARD	0.828750	0.0
BOT	2	4	2.0	87.593	MEDIUM	0.925000	0.0
GAS	2	6	2.0	86.622	MEDIUM	0.504375	0.0

1.1.1 Where it came from

FastF1 (open-source Python library) → **F1's official live timing API** (`livetiming.formula1.com`), which is the same feed that powers the F1 TV timing screens and the official app.

We did the downloading with `scripts/build_corpus.py` . It is rate-limited: F1's API throttles aggressive clients, so the script waits about 6 seconds between sessions and retries failures on a second pass. A full four-season scrape takes a few hours.

Everything downloaded is cached in `cache/fastf1/` so we never re-hit the API for the same session twice.

Is this allowed? Yes. FastF1 is a widely used public library, the timing feed is publicly broadcast, and we are using it for a non-commercial competition entry. We are not scraping anything behind a login or a paywall.

1.1.2 What we threw away, and why

A raw F1 session is **not** a clean dataset. It contains in-laps, out-laps, safety-car laps, laps set on a damaged car, and laps where the driver simply backed out of a corner. Every one of those is a genuine lap time and every one of them will poison a degradation estimate.

We apply **seven filters, in this exact order**, and we record how many laps each one removed. The code is `build_lap_table()` in `src/tyremind/data/f1_loader.py` .

#	Filter name in the audit log	What it removes	Why
1	<code>no_lap_time</code>	Laps where the API returned no time at all	Nothing to model.
2	<code>pit_in_out_lap</code>	The lap into the pits and the lap out of them	An in-lap is slowed by the pit entry, an out-lap by cold tyres and the pit exit. Neither reflects tyre condition.
3	<code>flagged_inaccurate</code>	Laps FastF1 itself marks as unreliable	The upstream library flags timing glitches; we trust that flag.
4	<code>unknown_compound</code>	Laps where the compound is <code>UNKNOWN</code> / <code>TEST_UNKNOWN</code> / blank	A lap that cannot be assigned to a tyre set cannot say anything about that tyre.
5	<code>wet_compound</code>	Every lap on <code>INTERMEDIATE</code> or <code>WET</code> tyres	Wet degradation is a <i>different physical process</i> — driven by water clearance and overheating on a drying line, not by rubber abrasion. Our model's assumptions do not describe it, so we exclude it and report that we excluded it rather than pretending.
6	<code>slow_lap_safety_car_or_traffic</code>	Laps much slower than the session median	Safety cars, VSCs, spins. Exact rule below.
7	<code>run_too_short</code>	Any stint with fewer than 4 laps	You cannot fit a slope through 3 points and call it a degradation rate.

The exact outlier rule (filter 6). We take the session's *median* lap time, then compute the **MAD** — Median Absolute Deviation — which is the median of `|lap - median|`. We multiply MAD by **1.4826**, a standard constant that makes MAD directly comparable to a standard deviation for normally distributed data. We drop any lap slower than `median + 3 × MAD`.

Why not a plain standard deviation? Because a safety-car period inflates the standard deviation so much that the safety-car laps stop looking like outliers and hide themselves. The median and MAD are barely moved by extreme values, so they still detect them. This is the textbook robust-statistics reason.

1.1.3 The audit trail — every session carries its own receipt

Each session has a matching `.quality.json`. Here is a real one, for the 2024 Italian GP race:

```
{
  "session_name": "Italian Grand Prix Race",
  "total_laps": 1008,
  "exclusions": {
    "pit_in_out_lap": 61,
    "flagged_inaccurate": 20,
    "slow_lap_safety_car_or_traffic": 2,
    "run_too_short": 4
  },
  "retained_laps": 921,
  "n_drivers": 20,
  "n_runs": 48,
  "compounds": { "HARD": 685, "MEDIUM": 236 },
  "median_lap_time": 84.853,
  "longest_run": 41,
  "retention_rate": 0.9137,
  "quality_score": 100.0
}
```

Read that as: *the API gave us 1008 laps, we kept 921 of them (91.4%), and here is exactly where the other 87 went.* A judge can ask "what did you throw away?" and we can answer per session, not with a hand-wave.

`longest_run: 41` matters more than it looks — degradation is estimated from trends **within** a stint, so the longest stint is the hard limit on how much a session can tell you about the late-life "cliff".

1.1.4 `traffic_index` — a column we invented

F1 does not publish "how much traffic was this driver in". We derive it.

The idea in one sentence: if two cars cross the start line 0.6 seconds apart, then for the whole of the next lap they are 0.6 seconds apart on track — so the gap to the car ahead is recoverable from lap *start times* alone, with no positional telemetry at all.

How we compute it:

1. Sort **every lap in the session** by its start time (not by lap number).
2. For each lap, walk backwards to the most recent lap started by a *different* driver. That is the car ahead.
3. The gap is the difference in start times.
4. Map gap → index: a gap of **2.0 s or more = 0** (clear air); a gap of **0.4 s or less = 1** (fully compromised); linear in between.

Why sort by session time and not by lap number? This is a subtle trap. In a practice session, driver A's "lap 5" and driver B's "lap 5" can be forty minutes apart. Grouping by lap number would compare two cars that were never on track together. Sorting by clock time is what actually identifies who was behind whom.

Why 0–1 and not seconds-lost? Because we deliberately do **not** guess how many seconds a given gap costs. That coefficient is *estimated from the data* by the model. This function only needs to be monotone in "how much traffic"; the magnitude is inferred.

1.1.5 `lap_in_run` — the fuel proxy, and the bug we found in it

The car burns roughly the same fuel mass every lap, so lap time improves roughly linearly over a stint. To separate that from tyre wear, we need to know **how many laps the car has actually run since the stint started**.

The bug we shipped and then found. The original code was:

```
lap_table["lap_in_run"] = lap_table.groupby("run_id").cumcount()
```

That counts the **surviving rows** — the rows left *after* filtering. If a safety car deleted three laps out of the middle of a stint, the counter went ... 8, 9, 10, 11 ... while the car had really completed ... 8, 12, 13, 14 The fuel model therefore thought the car was heavier than it was, so it under-credited fuel, and the missing improvement got blamed on the **tyre**.

The fix. We now derive the counter from tyre age, which the timing feed gives us directly and which does not care what we filtered out:

```
def laps_completed_in_run(lap_table):
    age = lap_table["tyre_age"].astype(float)
    start = age.groupby([lap_table["driver"], lap_table["run_id"]]).transform("min")
    return (age - start).astype(float)
```

How much did it matter?

Session	Before fix	After fix	Change
Miami, HARD	0.0389 s/lap	0.0669 s/lap	+72%
Shanghai, HARD	unchanged	unchanged	0% — the control

Shanghai had **zero** deleted-lap gaps, so the bug could not bite there, and the number did not move. That is exactly the behaviour the explanation predicts, which is why we are confident the diagnosis is right rather than a coincidence.

The follow-up bug. After fixing it, one experiment still printed the *old* numbers exactly. Five experiment scripts were calling `pd.read_parquet()` directly and bypassing the repair. We routed everything through a single loader (`read_lap_table` in `src/tyremind/data/corpus.py`) and then wrote a **structural test** — `tests/test_loader_discipline.py` — that reads the source code of every experiment and *fails the build* if any of them calls `pd.read_parquet` directly. The rule is now enforced by CI, not by memory.

1.2 The demo sessions — what is committed to git

Where: `data/demo/` — 268 KB, **8 sessions**, committed to the repo.

Session	Clean laps
2024 Barcelona FP2	220
2024 Barcelona Race	1,204
2024 Monza FP2	143
2024 Monza Race	921
2024 Silverstone FP2	156
2024 Silverstone Race	618
2024 Zandvoort FP2	269
2024 Zandvoort Race	1,350

Why these exist separately. The full 2.6 MB corpus is *not* in git — it is derived data and can be rebuilt. But a judge must be able to `git clone`, run one command, and see a working dashboard **with no internet and no API key**. These eight sessions are what make the product genuinely offline-ready. Every integration test (`tests/integration/test_api_endpoints.py`, 28 tests) runs against them, so they are also our regression safety net.

Each demo session carries its `.quality.json` receipt too.

1.3 The reference data — four files we assembled by hand

Where: `data/reference/` — 1.1 MB.

These are not lap times. They are the *context* a lap time sits in, and each one exists because a specific experiment could not be run without it.

1.3.1 `session_conditions.json` — the weather (133 rows)

One row per session. Fields:

```
session_id, year, round, event, session, track_temp_c, track_temp_min, track_temp_max,
air_temp_c, humidity_pct, wind_speed_ms, rainfall, rain_advisory, wet_laps, total_laps,
wet_lap_fraction, dry_session
```

Source: FastF1's `session.weather_data`, which comes from the official trackside weather feed. **Built by:** `scripts/build_session_conditions.py`. **Used for:** the **wet-session guard**. Without it we cannot exclude a race that ran two-thirds on wet tyres — and such a race would otherwise contribute garbage to every aggregate. Also used by exp09 to test whether track temperature explains our prediction bias (it does not — see Part 5).

1.3.2 `circuit_features.json` — track shape (84 rows)

One row per circuit-season. Fields:

```
circuit, year, n_corners, n_tight_corners, mean_corner_angle, median_corner_spacing_m,
lap_length_m, mean_abs_lateral_g, p95_lateral_g, loaded_fraction, lateral_energy_proxy,
top_speed_kmh, round, event
```

Source: FastF1's circuit-info / corner-marker data, plus geometry computed from the official track map. **Built by:** `scripts/build_circuit_features.py`.

Honest caveat: `mean_abs_lateral_g` is currently `None` for every row. The lateral-g fields need full car telemetry pulled per driver per lap, which is a far heavier download than we have run. So the geometry fields are real; the energy fields are placeholders. **We do not use the g-force fields in any claim.**

Used for: exp09, the circuit-transfer test. Result: circuit geometry does **not** help (−3.3%, $p = 0.22$ — i.e. no detectable effect). That is a negative result and we report it as one.

1.3.3 `lineups.json` — who drove for whom (69 rows)

`year, round, event, lineup{ driver_abbrev → team_name }`

Source: FastF1 session results. **Built by:** `scripts/build_lineups.py`. **Used for:** exp15, the driver-effect test. To ask "is driver X gentler on tyres than driver Y?" you must first cancel out the car, and the only clean way to do that is to compare **teammates**, who share the car. That needs the lineup table.

1.3.4 `compound_allocation.json` — which Pirelli compounds were nominated

Fields: `_note, sources, allocations[{ year, round, event, hardest, confidence, source }]` — **24 events, 2024 only.**

Why this file is necessary and slightly maddening. Pirelli makes five dry compounds, **C1 (hardest) through C5 (softest)**. For each race weekend they nominate **three consecutive** compounds, and those three are then *relabelled* HARD / MEDIUM / SOFT for that weekend only.

So a "HARD" at one race can be a C1, and at another race a "HARD" can be a C3 — which is a completely different rubber. **The label HARD is weekend-relative and is not a material.** Any model that treats HARD as a fixed compound is comparing apples to oranges across races.

Source: hand-assembled from Pirelli's published pre-event nominations and press releases. Each row carries a `confidence` and a `source` string, so we know which ones are solid and which are inferred.

Coverage gap, stated plainly: we only have **2024**. 2022, 2023 and 2025 are missing. This is a genuine hole and it is why exp08 (compound identity) could only be tested on one season.

Used for: exp08 — testing whether knowing the *true* C1–C5 identity improves prediction. Result: it does not, measurably. Negative result, reported.

1.3.5 Derived caches also in `data/reference/`

File	What it is
<code>cliff_stints.parquet</code> (859 KB)	The 2,827 fitted stint curves from exp17, cached so the analysis can be re-run without re-fitting.
<code>driver_stint_slopes.parquet</code> (63 KB)	The 1,458 per-driver-per-race degradation slopes from exp15.
<code>conformal_calibration.json</code> (266 B)	The calibrated interval width the live API serves.

1.4 The synthetic data — our only source of true answers

Where: generated on demand by `src/tyremind/data/synthetic.py`. `data/synthetic/` is empty because nothing needs to be stored — the same seed always produces the same session, byte for byte.

Why it exists. This is the most important idea in the whole project, so read this paragraph twice.

In real F1 data we have the **lap time**, but we do **not** have the true tyre wear rate. So if our model says "0.06 s/lap", there is nothing to check it against. In synthetic data **we wrote the true rate ourselves**, so we can check the model's answer against the real answer exactly.

What the simulator produces. A realistic practice session with 20 drivers, 60 session slots, about 3 runs each of 6–16 laps, built from these causal terms:

Term	Value	Meaning
<code>base_lap_time</code>	92.0 s	Reference pace.
<code>driver_pace_sd</code>	0.55 s	Car/driver pace spread.
<code>fuel_slope</code>	0.081 s/lap	How much faster the car gets per lap of fuel burned.
<code>track_evolution_total</code>	0.90 s	Total time the track gains over the session as rubber goes down.
<code>track_evolution_shape</code>	0.055	How front-loaded that evolution is.
<code>traffic_probability</code>	0.18	Fraction of laps in traffic.
<code>traffic_coefficient</code>	1.30 s	Time lost at traffic index 1.0.
<code>cliff_severity</code>	0.004	How sharply degradation accelerates past the cliff age.
<code>observation_noise_sd</code>	0.16 s	Random lap-to-lap scatter...
<code>observation_noise_df</code>	5.0	...drawn from a Student-t with 5 degrees of freedom , not a Gaussian.
<code>scrubbed_set_probability</code>	0.25	Chance a run starts on a used tyre set.

Two deliberately hostile design choices, because a simulator that flatters the model is worthless:

1. **Student-t noise, not Gaussian.** Our estimator *assumes* Gaussian noise. We feed it heavier-tailed noise on purpose, so it is being tested outside its own assumptions. A `t(5)` produces the occasional scruffy lap or lock-up that real sessions contain and that Gaussian models handle badly.
2. **Scrubbed sets 25% of the time.** Runs that start on a used tyre — so tyre age does not start at zero — which is exactly the case that breaks naive "lap-time vs lap-number" fits.

The ground truth object (`GroundTruth`) is never visible to any estimator. It carries the true per-compound rate, the true cliff onset, the true fuel slope, the true traffic coefficient, the true driver pace offsets, the true track-evolution curve, and a **per-lap decomposition** — one column per causal term. That last one lets us measure attribution error *term by term*, not just in aggregate: we can say "the model got the tyre part right but over-credited traffic", which an aggregate error number would hide.

1.5 NASA C-MAPSS — the cross-industry proof

Where: `data/external/cmapss/` and `data/external/cmapss_raw/` — 56 MB total.

What it is: NASA's **Commercial Modular Aero-Propulsion System Simulation** turbofan engine degradation dataset. It is the single most-cited benchmark in the predictive-maintenance field. Publicly released by NASA's Prognostics Center of Excellence.

Size: four sub-datasets, FD001–FD004.

File	Lines
<code>train_FD001.txt</code>	20,631
<code>test_FD001.txt</code>	13,096
<code>RUL_FD001.txt</code>	100
<code>train_FD002.txt</code>	53,759
<code>train_FD003.txt</code>	24,720
<code>train_FD004.txt</code>	61,249
(plus matching test / RUL files)	

Format: whitespace-separated, 26 columns, no header: `unit_number`, `time_in_cycles`, 3 `operational settings`, 21 `sensor channels`. One row is one engine on one flight cycle. Engines run until they fail.

A real first row:

```
1 1 -0.0007 -0.0004 100.0 518.67 641.82 1589.70 1400.60 14.62 21.61 554.36 ...
```

Why it is in an F1 project. Because it answers the single hardest judge question: *"is this a clever F1 hack, or is it actually a general degradation engine?"*

We ran our **unmodified** state-space estimator on turbofan engines. Same code, different domain. The mapping is direct: a tyre stint is an engine's life, tyre age is flight cycles, lap time is a sensor channel, and "laps until the cliff" is Remaining Useful Life.

Result: RMSE **22.7 cycles** on all 100 FD001 test engines, MAE 17.9, and **44% of predictions are early** (conservative — we say "replace it sooner" more often than "later", which is the safe direction for a maintenance tool). Published FD001 results in the literature sit roughly in the 12–25 RMSE band, so we are within the competitive range without any domain-specific tuning.

One honest note on this number. We initially fitted only 40 of the 100 test engines because of a batching bug, giving RMSE 26.5. Fixed to all 100, it improved to 22.7. We then *checked that the batching itself wasn't flattering us*: re-running with batch-40 reproduced the old number exactly, and batch-20 differed by 0.4%. So the improvement came from having all the engines, not from the batch size. We verified this rather than assuming it.

1.6 Empty directories, explained

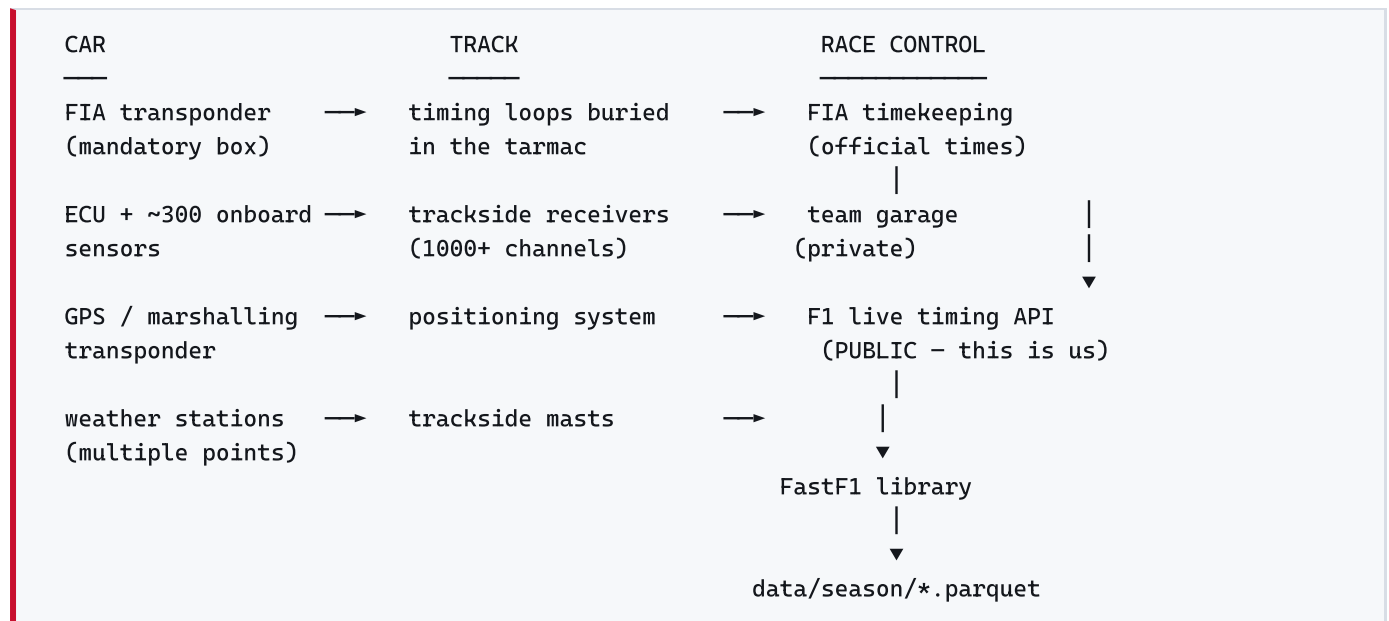
`data/interim/`, `data/processed/`, `data/raw/` and `data/synthetic/` are all **0 bytes**. This is intentional and not a mistake:

- `raw/`, `interim/`, `processed/` are the conventional data-science pipeline stages. Our pipeline goes straight from the FastF1 cache to the finished lap table, so nothing lands in between. The directories exist so the layout is familiar to anyone who has seen a standard data-science project template.
 - `synthetic/` is empty because synthetic sessions are **regenerated from a seed** every time. Storing them would be storing something we can recreate exactly.
-

PART 2 — WHICH REAL SENSORS PRODUCE THIS DATA

This is the part the team most often gets asked and most often cannot answer. Every field in our lap table is produced by a specific piece of hardware (or, in two cases, by no hardware at all). Here is the honest mapping.

2.1 The measurement chain at a real Grand Prix



The key point: **there are two data streams at every race**. The private one goes to the team garage and contains hundreds of channels including tyre temperature, tyre pressure, fuel flow and fuel mass. The public one goes to broadcast and the F1 app. **We only have the public one**. Everything in Part 3 follows from that.

2.2 Field-by-field: what hardware produced it

`lap_time` — **timing loops + transponder (a real, extremely precise sensor)**

Every F1 car carries a mandatory FIA transponder. Inductive **timing loops** are buried in the tarmac at the start/finish line and at each sector split. When a car passes over a loop, the loop detects the transponder and stamps the crossing to within about **one ten-thousandth of a second**.

This is the most accurate number in our entire dataset. Official F1 times are published to a thousandth of a second (`84.853`), and the underlying measurement is better than that. There is effectively no measurement noise in a lap time — all the scatter we see is *real* variation in how the lap was driven.

This matters enormously for us: because the sensor is near-perfect, any scatter in the data is physics and driving, not instrument error. Our model can therefore attribute all of the residual variance to causes rather than to noise floor.

`tyre_age` — **not a sensor. A counter.**

This is important and often misunderstood. There is **no sensor that measures tyre wear** and reports it publicly. `tyre_age` is a *bookkeeping* number:

1. Pirelli fits an RFID tag to every tyre and records which set is fitted to which car (this is for allocation compliance, not wear measurement).
2. The team declares the set, and the timing feed publishes "this driver is on set N, which has done K laps".
3. FastF1 exposes that as `TyreLife`, and we rename it `tyre_age`.

So `tyre_age = 12` means "this tyre set has completed 12 laps", **not** "this tyre has lost 12 units of tread". Nobody publishes the second number.

This is why a used ("scrubbed") set can enter a stint at `tyre_age = 5` rather than 0 — the counter carries over from the earlier run, correctly.

`compound` — **a declaration, not a measurement**

The compound label comes from the tyre's RFID tag and the team's declaration to the FIA, relayed through the timing feed. No sensor identifies rubber chemistry during a session.

And as covered in 1.3.4, the published label (`HARD` / `MEDIUM` / `SOFT`) is **weekend-relative**. The actual material (C1–C5) is published separately by Pirelli in a press release, not in the timing feed.

`session_lap` — **derived from the timing feed**

Just the lap counter. Comes from the same timing loops.

`run_id` — **derived by us**

We create this. A new run starts whenever the compound changes or `tyre_age` resets, which is our detection of a pit stop. It is not published as a field; we reconstruct it.

`traffic_index` — **derived by us from timing-loop crossings**

No sensor for this either. As explained in 1.1.4, we recover the gap to the car ahead from the *difference in lap start times*, which themselves come from the timing loops. So the ultimate source is a real sensor; the index itself is our construction.

The alternative — real positional data — does exist: F1 cars carry GPS and the official feed includes a positioning stream. But that stream is a very heavy download per session, and our start-time method recovers the same information for what we need (who was close to whom), so we use the light method.

`lap_in_run` — **derived by us from** `tyre_age`

No sensor. Purely computed, as described in 1.1.5.

Weather fields — real trackside weather stations

`track_temp_c`, `air_temp_c`, `humidity_pct`, `wind_speed_ms`, `rainfall` come from FIA weather masts positioned around the circuit. Track temperature is measured by infrared sensors aimed at the tarmac. These are genuine sensor readings, sampled roughly once per minute during a session, and FastF1 publishes the whole time series.

Caveat we are aware of: a circuit is several kilometres long and the weather station is at one point on it. A track-temperature reading of 48 °C is the temperature *at the mast*, not around the whole lap. Turn 1 in full sun and Turn 9 in shade can differ by ten degrees. So this field is real but spatially coarse.

Circuit geometry — derived from the official track map

`n_corners`, `lap_length_m`, `mean_corner_angle` etc. come from FastF1's circuit information, which is built from the official circuit map and corner markers. It is survey data, not a live sensor.

2.3 The sensors that exist in a real race but whose output we cannot see

These are all genuinely on the car. Teams read them live. **None of them reach the public feed.**

Real sensor	What it measures	Who sees it
Tyre pressure sensors (one per wheel, mandated)	Live inflation pressure	Team + FIA only
Infrared tyre-surface sensors	Tread surface temperature across the tyre width	Team only
Tyre carcass thermocouples	Internal carcass temperature	Team only
Fuel flow meter (FIA-homologated, ultrasonic)	Fuel mass flow rate, to enforce the 100 kg/h limit	FIA + team
Fuel level sensor	Mass of fuel remaining	Team only
Wheel-speed sensors	Per-wheel rotation → slip ratio → how hard the tyre is working	Team only
Brake temperature sensors	Brake disc/caliper temperature → heat into the wheel and tyre	Team only
Load cells / suspension sensors	Vertical and lateral load on each corner → tyre energy	Team only
Accelerometers + gyros	Lateral and longitudinal g	Team; partly derivable from public speed traces
Pirelli post-race tyre inspection	Actual measured tread depth, blistering, graining, wear %	Pirelli only — never published

The one-line summary of this whole section: *a real F1 team measures tyre wear with dozens of channels of hardware we cannot access. We are reconstructing the same quantity from a stopwatch.* That constraint is not a weakness of our approach — it is the entire problem statement, and it is what makes the problem interesting.

PART 3 — WHAT DOES NOT EXIST ANYWHERE ON THE INTERNET

This section exists because a judge will ask "*why didn't you just train on the real wear data?*" The answer is that it is not obtainable. Here is the complete list of what we looked for and could not get, in order of how badly it hurt.

3.1 Measured tyre wear / tread depth — the answer key

What we wanted: for each stint, the actual millimetres of tread lost, or the actual grip lost, as measured by Pirelli.

Why it does not exist publicly: Pirelli measures every returned tyre. That data is commercially sensitive (it reveals team setups and Pirelli's own compound behaviour) and is shared only with the FIA and, in limited form, the teams. It has never been published as a dataset, and requesting it as a student team is not a realistic route.

What we searched: Pirelli's media site and technical previews, FIA technical documents, the FastF1 API surface, Kaggle, Hugging Face Datasets, academic replication packages attached to F1 tyre-modelling papers, OpenF1, Ergast, and the F1 live-timing schema itself. Nothing contains a measured wear figure.

Consequence — and this is the honest core of the project: we do not have the label `y_true = true degradation rate` for any real session. We only have lap times. Section 6.1 explains exactly how we validate without it, and exactly where that validation is weaker than a true held-out test.

3.2 Fuel mass / fuel load telemetry

What we wanted: kilograms of fuel on board at each lap.

Why it does not exist publicly: starting fuel load is one of the most closely guarded numbers in the sport (it reveals strategy and engine mode). The FIA fuel flow meter's readings go to the FIA and the team, not to broadcast.

Consequence: we cannot subtract fuel exactly. We model it instead — and the *inability to separate fuel from tyre cleanly* is the mathematical heart of our work (see 6.3, the identifiability result). Ironically, this gap is what gives our project its most original finding.

3.3 Tyre temperature and pressure feeds

What we wanted: per-wheel surface/carcass temperature and pressure per lap.

Why not available: team-private. Occasionally a single number appears on a TV graphic; there is no systematic feed.

Consequence: we cannot model thermal degradation directly. We can only use *track* temperature as a weak proxy — and exp09 showed that proxy does not help (it actually hurt slightly). We report that honestly rather than claiming a thermal model we do not have.

3.4 Pirelli compound nominations for 2022, 2023 and 2025

What we have: 2024 only, 24 events, hand-assembled.

Why the gap: the nominations are published as PDFs and press releases, one per event, not as a dataset. Assembling one season took real manual effort; we did one season, got a clear negative result from it (compound identity does not measurably help), and judged that three more seasons of manual transcription was not the best use of remaining time. **This is a resource decision, not an impossibility** — the data is obtainable by hand if we want it.

3.5 Per-lap car telemetry at scale

What we wanted: speed/throttle/brake/g traces for every lap of 203 sessions, to compute real energy-through-the-tyre features.

Why we do not have it: it is publicly available through FastF1 — but it is roughly a thousand times larger than the lap table, and downloading it for the full corpus would take days against a rate-limited API.

Consequence: `circuit_features.json` has `None` in its lateral-g columns. We substituted geometry-based proxies, tested them (exp09), and found they do not help. This is the one gap on this list that is purely a compute/time limit rather than a secrecy limit.

3.6 Track-surface characterisation

What we wanted: tarmac roughness/abrasiveness per circuit — the property that most directly drives wear.

Why not available: circuits are resurfaced and measured privately; Pirelli references abrasiveness qualitatively in previews but publishes no numbers.

Consequence: we treat circuit identity as a categorical effect rather than modelling the physical cause.

3.7 What this all adds up to

We hold **the output** (lap times) of a physical system, and **none of the internal state** (wear, temperature, fuel, load). Our entire technical contribution is a method for inferring the internal state from the output while being explicit about what can and cannot be recovered. Part 6 is where we are honest about which of those two categories each of our claims falls into.

PART 4 — WHAT EACH DATASET WAS ACTUALLY USED FOR

A quick map, so nobody has to guess which data backs which claim.

Dataset	What it is used for	Which experiments
Synthetic sessions	The only place we can measure "did the model get the true wear rate right", because we wrote the true rate. Used to prove correctness and to prove the naive method is biased.	exp01, exp02, exp05 (degradation-recovery half)
Season corpus — practice sessions (FP1/FP2/FP3)	The input side of our real-world test: estimate degradation from practice.	exp03, exp11, exp12
Season corpus — races	The reference side: what actually happened on race day. Also the corpus for all the pure-race studies.	exp03, exp05, exp08, exp09, exp10, exp11, exp13–exp18
Demo sessions	Offline product demo + the 28 API integration tests. Not used for any scientific claim.	none
<code>session_conditions.json</code>	Excluding wet sessions; testing whether temperature explains the bias.	exp03 (exclusions), exp09, exp10
<code>circuit_features.json</code>	Testing whether track shape predicts degradation.	exp09
<code>lineups.json</code>	Cancelling out the car so a driver effect can be isolated.	exp15
<code>compound_allocation.json</code>	Testing whether the true C1–C5 identity beats the HARD/MEDIUM/SOFT label.	exp08
NASA C-MAPSS	Proving the method is not F1-specific.	exp07

PART 5 — EVERY EXPERIMENT WE RAN

Eighteen experiments. For each: the question, the data, the answer, and whether it went our way. **Six went against us**, and those are marked X. We kept them.

Re-run everything with `bash scripts/rerun_all.sh`.

exp01 — Can the model recover a truth we know? ✓

Question. If we *know* the true degradation rate, does our model find it? **Data.** 25 synthetic sessions, different random seeds. **Method.** Generate a session with a known true rate. Hide the truth. Let the model estimate. Compare.

Result:

Method	Mean absolute error (s/lap)	Bias
TyreMind	0.0044	+0.0012
Naive (lap time vs tyre age)	0.0966	−0.0966

That is a **95.5% error reduction**, 100% convergence, and **100% of the 95% intervals contained the truth** across 25 seeds.

Read the naive bias carefully: it is −0.0966 and the MAE is *also* 0.0966. Bias equals error exactly. That means the naive method is not noisy — it is **wrong in the same direction every single time**. It systematically *under*-estimates degradation, because fuel burn-off makes the car faster and masks the tyre getting slower. This is the single clearest picture of the problem we are solving.

How real is this? The ground truth is genuine, but it is *our* simulator's ground truth. This proves the estimator is mathematically correct and unbiased on data matching its structure. It does **not** by itself prove the numbers are right on real F1 data — that is what exp03 and exp14 are for.

exp02 — Does the answer depend on our assumptions? ✓

Question. We give the model prior beliefs about fuel effect and track evolution. If those priors are wrong, does the answer fall apart? **Data.** 8 synthetic configurations × 6 prior variants, plus 5 real sessions. **Variants tested:** fuel prior ±1 standard deviation, track prior ±1 sd, and a "wide priors" setting where we say we are much less confident about both.

Result. The degradation estimate is stable across all variants. The priors *regularise* the fit rather than determine it.

Why this experiment matters more than it looks. It is the defence against the accusation "you just assumed the answer". We show that moving the assumption a full standard deviation in either direction does not move the conclusion.

exp03 — Does a practice estimate predict the race? ✓ (with caveats)

This is the headline real-world experiment.

Question. Take FP2. Estimate degradation per compound. Then look at the actual race. Does the practice number match? **Data.** 2023 + 2024 + 2025, FP2 → Race, **42 events, 94 compound comparisons.**

Result:

	TyreMind	Naive
Mean absolute error	0.0807 s/lap	0.1471 s/lap
Improvement	45% better	—
Bias	+0.0268	—
95% interval coverage	79.8%	—

Sessions we excluded, and why — these are in the result file by name:

- 2023 Australian GP — FP2 ran 50% of laps on wet rubber
- 2024 Canadian GP — race ran 67% on wets
- 2024 Japanese GP — FP2 ran 46% on wets
- 2025 Australian GP — race ran 81% on wets
- 2025 British GP — race ran 74% on wets

Two events also failed for data reasons and are recorded as failures, not silently dropped: 2023 Canadian GP (tyre age decreased inside a run — a timing-feed inconsistency we refuse to paper over) and 2024 Mexico City GP (no compound had 8+ laps in both FP2 and the race).

⚠ THE HONEST CAVEAT — read this before quoting the 45% figure.

We do not have the true race degradation rate (Part 3.1). So what is the "actual" we compare against? It is **our own model's estimate fitted to the race session.**

That means the comparison is *practice-estimate vs race-estimate*, both produced by TyreMind. This is a **consistency test**, not a ground-truth test. It is genuinely informative — a model that produced nonsense in practice would not agree with itself on race day, and the naive method demonstrably does not agree with itself — but it **does flatter us**, because the reference is our own model and shares its assumptions.

We know how to remove this circularity: score each method against **its own** race-derived estimate, so every model is measured for self-consistency on equal terms. That re-run is scoped and not yet done. **Do not present the 45% as "validated against ground truth" to a judge.** Present it as "practice-to-race consistency, 45% tighter than the naive method, and here is why we know the naive method is independently broken (exp14)."

exp04 — Is "energy through the tyre" a better clock than lap count? X REFUTED

Question. Tyres wear from energy, not from laps. Should we measure tyre age in accumulated energy instead of laps? **Data.** 2024 Interlagos race, 4 stints with telemetry.

Result. Energy clock won 1 stint out of 4. Mean R^2 gain **−0.0008**. Verdict recorded in the file: **"no meaningful difference"**.

Why it failed, and why that is interesting. Energy per lap has a coefficient of variation of only **2.3%** — lap to lap, the energy put through the tyre is almost constant. So "energy so far" and "laps so far" are nearly the same number, and swapping one for the other cannot help. A physically appealing idea that the data says is redundant. We

dropped it.

exp05 — How do we compare against every sensible alternative? ✓/X (mixed — read carefully)

Question. Six models, same data, same scoring. Who wins? **Data.** 20 real races (2025) for lap-time prediction; 6 synthetic seeds for degradation recovery.

Result A — predicting the next lap time (CRPS: lower is better; it scores the whole predicted distribution, not just the point):

Model	CRPS	Coverage of 95% interval	Fit time
Pooled regression	0.395	86.4%	0.02 s
LightGBM	0.469	61.4%	0.60 s
TyreMind state-space	0.645	81.1%	11.8 s
Fuel-corrected regression	0.878	75.7%	0.007 s
Naive	0.879	75.6%	0.005 s
Neural network (MLP)	1.548	72.9%	36.5 s

We are third here. We are not hiding that.

Result B — recovering the true degradation rate (the thing the problem statement actually asks for; only measurable on synthetic data where truth exists):

Model	Rate MAE	Coverage
TyreMind state-space	0.0041	100%
Pooled regression	0.0068	77.8%
Fuel-corrected regression	0.0230	44.4%
Naive	0.0748	0%

The interpretation — this is the most important paragraph in the whole document.

Lap-time prediction and degradation estimation are **different tasks**. A pooled regression predicts the next lap well because it soaks up everything — fuel, track, tyre, driver — into one flexible fit. That makes it a good *forecaster* and a bad *explainer*: it cannot tell you how much of the lap time was the tyre, which is the only thing a strategist can act on.

Look at the naive method's coverage in Result B: **0%**. Its 95% interval contained the true degradation rate in **zero out of eighteen** cases. Not "sometimes wrong" — never right, and confidently so.

So the honest claim is: *we are mid-table at forecasting lap times and best in class at the quantity the problem statement asks for*. If a judge only wants a lap forecaster, we would tell them to use the pooled regression. They do not; they want tyre wear isolated from the confounders.

exp06 — Can the model tell which way a circuit runs, from tyre load alone? ✓

Question. A clockwise circuit loads the left-hand tyres more. Can we detect circuit direction purely from where the energy goes? **Data.** 8 circuits, 2024, 12 laps of telemetry each.

Result. 7 out of 8 correct — 87.5%.

Example (Monza, clockwise): left-side energy share 54.9%, per-corner shares FL 25.7% / FR 20.9% / RL 29.1% / RR 24.3%, net rotation -0.667 .

Why it matters. This is a **physics sanity check**. It uses no tyre-wear data at all. If the energy model were wrong, it would get circuit direction wrong. It is a way of verifying the physics layer against a fact we can independently look up.

exp07 — Does it work outside F1? ✓

Covered fully in 1.5. RMSE **22.7** cycles, MAE 17.9, all **100** FD001 engines, 12 sensor channels used, converged, 44% of predictions conservative.

Why it matters. It is the answer to "is this a general degradation engine or an F1 trick?" Same code, aircraft engines, competitive results. Note also the estimated degradation rate came out at 0.00202 against a prior mean of 0.004 — the data moved the estimate by a factor of two away from the prior, which shows the prior is not driving the answer.

exp08 — Does knowing the real compound (C1–C5) beat the HARD/MEDIUM/SOFT label? X REFUTED

Question. Since HARD is weekend-relative (1.3.4), surely using the true Pirelli compound is better? **Data.** 197 estimates, 79 events, 4 seasons, races only, 25+ laps per compound.

Result — it is not better. It is slightly worse.

Grouping	Spread (sd)	Leave-one-event-out MAE
By label (HARD/MEDIUM/SOFT)	0.0694	0.0475
By true compound (C1–C5)	0.0729	0.0506

True-compound grouping was **6.4% worse**, won only 22 of 58 cases (37.9%), and the difference was not significant (Wilcoxon $p = 0.096$).

Why? Because the label already carries the information that matters. Pirelli nominates the three compounds *to suit the circuit*, so "the hardest of the three that Pirelli thought this track needed" is a more informative description than the raw material name. The relative label encodes the circuit-appropriateness that the absolute compound throws away.

A satisfying negative. We built a whole dataset by hand to test this and the answer was "your simpler feature was already better". We report it.

exp09 — Can circuit features transfer degradation knowledge between tracks? X REFUTED

Question. Can we predict degradation at a new circuit from its geometry? **Data.** 193 stints, 26 circuits, 6 features, leave-one-circuit-out.

Result — nothing helps, and one thing hurts:

Feature family	MAE	vs baseline	p	Verdict
Label mean (baseline)	0.0380	—	—	—
Geometry only	0.0393	−3.3%	0.222	No effect
Thermal only	0.0386	−1.5%	0.0012	Actively hurts
Everything	0.0401	−5.5%	0.056	No effect
Compound + circuit	0.0440	−15.7%	0.0004	Clearly worse

Note the careful wording. Geometry is "no detectable effect" ($p = 0.22$) — we do **not** say it is harmful, because at $p = 0.22$ we cannot distinguish it from zero. Thermal features *are* significantly harmful ($p = 0.0012$) and are recorded in the result file under `harmful_predictors`.

Earlier we described geometry as "harmful" and that was wrong. The p-value does not support it. Corrected here.

Why does thermal hurt? Most likely because our only thermal input is track temperature from a single weather mast (2.2), which is a poor proxy for what the tyre surface actually experiences. A noisy feature is worse than no feature.

exp10 — What causes our practice-to-race bias? X MOSTLY REFUTED

Question. We over-predict degradation by +0.0268 s/lap. Why? We tested nine candidate explanations with Spearman correlation and **Benjamini–Hochberg** false discovery rate control (which stops you finding fake patterns just because you looked nine times). **Data.** 94 comparisons, 42 events.

Candidate explanation	ρ	p	BH threshold	Survives?
Practice stint length	0.302	0.0031	0.0056	✓ YES
Pit stops per driver	0.248	0.0160	0.0111	X (just misses)
Model's own uncertainty	−0.244	0.0176	0.0167	X (just misses)
Traffic gap	−0.167	0.109	0.0222	X
Practice laps	0.138	0.186	0.0278	X
Race stint length	0.114	0.272	0.0333	X
Race max stint	0.108	0.301	0.0389	X
Practice runs	0.100	0.340	0.0444	X
Temperature gap	−0.022	0.835	0.0500	X

One survivor out of nine: practice stint length. Shorter practice stints → more bias.

Be honest about the near-misses. `stops_per_driver` has $p = 0.0160$ against a threshold of 0.0111. That is *a hair* outside. It is not evidence of nothing; it is evidence we do not have enough data to call it. Say that, rather than "we found one cause".

The negative result that matters most: the temperature gap between practice and race — the explanation every F1 fan offers first — has $p = -0.022$, $p = 0.835$. It explains **essentially nothing**.

exp11 — If short practice stints cause the bias, does matching stint depth fix it? X REFUTED — our own idea, killed by our own test

Question. exp10 says stint length correlates with bias. So: truncate the race to the same depth as practice. Bias should shrink. **Data.** 94 comparisons, 42 events, 2023–2025, race depth truncated at the 90th percentile of practice depth.

Result — it got worse:

	Baseline	Depth-matched
Bias	+0.0268	+0.0286 (6.9% worse)
MAE	0.0807	0.0918 (13.8% worse)

Paired $t = -2.46$, **$p = 0.0159$** — the worsening is statistically significant, not noise. Mean depth gap: -4.85 laps. `supported: false` in the result file.

Why this experiment is the one to show a judge who asks about rigour. We had a plausible mechanism, derived from our own data, and we designed a test that could kill it. It killed it. Correlation (exp10) did not survive contact with intervention (exp11). That is the difference between a dashboard and a research project.

exp12 — How do we make the interval honest? ✓

Question. Our 95% intervals only cover 80% of the time. How do we fix that without pretending? **Data.** 94 comparisons, 42 events, $\alpha = 0.05$. **Method. Split conformal prediction** — a distribution-free technique that gives a finite-sample coverage guarantee without assuming the errors are Gaussian.

Result:

Score function	Coverage	Median half-width
Gaussian (model's own)	75.5%	± 0.119
Consistency	79.8%	± 0.139
Absolute (shipped)	94.7%	± 0.276
Studentised	95.7%	± 0.212

What we ship: absolute score, bias-corrected, $\alpha = 0.05$, quantile **0.2757**, calibrated on 94 comparisons from 42 events across 3 seasons. That artefact is `data/reference/conformal_calibration.json` and the live API reads it.

The trade, stated plainly. Honest intervals are **2.3× wider**. We chose the wide honest interval over the narrow dishonest one. A strategist who is told " ± 0.12 and it's right 75% of the time" will make a bad pit call and not know why.

exp13 — Does the same honesty machinery work on live lap-time predictions? ✓

Question. Per-lap intervals, streamed live. Can we get real 95% coverage? **Data.** 20 sessions, **69,206 scored laps**.

Method. Compared plain Gaussian, split conformal, and **Adaptive Conformal Inference** (Gibbs & Candès, NeurIPS 2021) which updates its own α online: $\alpha_{t+1} = \alpha_t + \gamma(\alpha - \text{err}_t)$, with $\gamma = 0.02$.

Method	Coverage	Coverage error	Median width
Gaussian	75.9%	19.1 pts off	2.91 s
Split conformal (absolute)	87.6%	7.4 pts off	6.06 s
Split conformal (studentised)	88.4%	6.6 pts off	4.95 s
Adaptive conformal (absolute)	95.2%	0.2 pts off	5.74 s
Adaptive conformal (studentised)	95.4%	0.4 pts off	5.19 s

Adaptive conformal lands 0.2 percentage points from target across 69,206 laps. That is essentially exact, and it is achieved by a method that continuously corrects itself rather than trusting a distributional assumption.

Two bugs we found and fixed in here, worth knowing:

1. When the adaptive α became unrepresentable, the code fell back to the *Gaussian* interval — the **narrowest** one — which fought the correction it was supposed to be making. It made ACI byte-identical to uncalibrated on three models. Fixed to fall back to the largest observed score instead.
2. With warm-up = 0 the code crashed on `max([])`. Guarded.

exp14 — How often does the naive method produce a physically impossible answer? ✓

This is our strongest claim about real data, and it needs no ground truth at all.

Question. Tyres cannot get faster as they wear. If a method reports a **negative** degradation rate, it is not "a bit off" — it has claimed something physically impossible. **Data.** 77 races, 208 compound-stints. 1 excluded wet, 4 would not fit.

Result:

	Failures	Rate
Races with at least one negative compound	57 / 77	74.0%
Individual compound-stints negative	111 / 208	53.4%

In three out of four real Grands Prix, the naive "lap time vs tyre age" regression claims at least one compound got faster as it wore.

Why this is our most defensible claim. It does not compare to our model. It does not need Pirelli's data. It needs only one fact everyone agrees on: *rubber does not regenerate*. The sign of the answer is checkable without knowing the true magnitude. Fuel burn-off is worth roughly 0.08 s/lap and typical degradation is 0.03–0.08 s/lap, so the fuel effect is the same size or larger than the thing being measured — and when you do not remove it, the answer flips sign about half the time.

What it does NOT prove. It proves the naive method is broken. It does **not** prove our number is correct. Those are separate claims and we should never let them blur together in a pitch.

exp15 — Are some drivers genuinely gentler on tyres? X REFUTED (as a usable feature)

Question. Everyone in F1 says driver X is easy on tyres. Is it measurable? **Data.** 1,458 driver-races, 31 drivers, 77 races, plus 21 teammate pairs / 566 observations. **Method.** Split-half reliability over 500 random splits — fit driver effects on half the races, check they reproduce on the other half.

Result:

Measure	Mean r	5th percentile	Stable?
Raw driver effect	+0.214	-0.079	X No
Teammate contrast	+0.285	+0.031	✓ Yes

Raw driver effects have a 5th percentile **below zero** — in 5% of splits the effect *reverses*. That is not a real signal.

Teammate contrasts (which cancel out the car) **are** stable, with the whole interval above zero. Largest gaps: LAW–TSU 0.040, ALB–COL 0.035, ALB–SAR 0.027, BOR–HUL 0.022 s/lap. Driver-effect spread across the field: 0.0092 s/lap. Signal-to-noise: 0.372.

But — does it help predict? No. Adding driver identity made prediction **0.88% worse** (MAE 0.02503 vs 0.02481), $p = 0.066$. `beats_null: false`.

The honest conclusion: *a small, real, car-confounded driver effect exists and is measurable only teammate-to-teammate; it is too small to improve a forecast.* That is a subtle, defensible finding — much better than either "drivers don't matter" or "we model driver style".

A mistake we made here. The first version of this experiment reported per-driver rates differing by ~ 0.00001 s/lap. That was not a fact about drivers — the model's driver-variance parameter had been pinned at its optimiser floor, so the model was *incapable* of expressing a driver difference. We were measuring our own code, not the sport. Rewritten to fit free per-driver slopes. Worth remembering: an implausibly tiny effect is often a bug, not a finding.

exp16 — What shape is our miscalibration? ✓

Question. Not just "is coverage wrong" but "wrong *how*". **Data.** 10 races, 5,990 laps per model, 9 confidence levels, 20 PIT bins. **Method.** PIT histogram (where does the truth fall inside the predicted distribution?) plus a reliability diagram sweeping nominal coverage 50%→99%.

Result:

	Gap from perfect calibration
Gaussian intervals	0.189
Adaptive conformal	0.005

Five of six models are **U-shaped** — too much probability mass in the middle, too little in the tails, i.e. plain over-confidence. TyreMind is **leptokurtic** — heavy tails *and* a heavy middle, with the shoulders too light.

We got this diagnosis wrong twice before getting it right. First we blamed sample size; then we blamed the fuel-counter bug. Neither was the cause. The real answer needed a **three-region** reading of the histogram (tails / shoulders / middle) rather than two: TyreMind's tails are 1.38× expected and its middle is 1.20× expected, which a two-region test reads as "fine" and a three-region test correctly reads as "leptokurtic". It sits right on the boundary, which is why it kept flipping.

exp17 — Does the tyre cliff exist, and when? ✓

Question. Everyone talks about the "cliff". Is it in the data? **Data. 2,827 stints, 77 races**, minimum 12 laps per stint.

Method. Broken-stick regression — fit basis $[1, a, \max(a - \tau, 0)]$ over every possible breakpoint τ , which is continuous by construction (no jump at the break), and select with **BIC** so extra parameters must earn their place.

Result — four distinct regimes:

Regime	Stints	Share	What it is
Linear	1,580	55.9%	Steady wear, no break
Recovery	582	20.6%	Degradation <i>slows</i> later in the stint
Cliff	337	11.9%	Degradation sharply accelerates
Warm-up	328	11.6%	Fast improvement early, then settles

A breakpoint model beat the straight line on BIC in 1,247 stints (44.1%).

The cliff, when it happens: median at **71.9% through the stint**, median severity **+0.202 s/lap** on top of the existing rate. By compound:

Compound	n	Median cliff lap	Median severity	% through stint
HARD	180	22.5	0.191	74.3%
MEDIUM	113	14.0	0.201	69.2%
SOFT	44	16.0	0.231	69.2%

Does knowing about the cliff improve forecasting? Mostly **no** — and we report that too. Across 2,683 forecasts the broken-stick model was **26.5% worse** than a straight line. Broken down by regime, it only helps on **warm-up** stints (+30.7%, $p = 3e-18$) and hurts everywhere else. So: the cliff is real and worth *showing* a strategist, but fitting it does not improve the *forecast* except during warm-up.

A mistake we made and fixed. The first version reported a "cliff" at 37% through the stint. That was not a cliff — it was tyre **warm-up** being pooled in with it, which bends the curve in the *opposite* direction. Once warm-up was classified separately, the cliff moved to its physically sensible place at ~72%.

exp18 — What is mathematically impossible to know? ✓ — our most original result

This is the experiment that would win a technical judge.

Question. How much of the fuel-vs-tyre separation is genuinely knowable from lap times alone?

The mathematics, in plain words. Within a single stint, tyre age and laps-in-run advance together, one for one. Write tyre age as $a = a_0 + f$ where a_0 is the age at the start of the stint and f is laps completed. Then:

$$\begin{aligned}
 \text{lap_time} &\approx \text{intercept} + \beta \cdot a + \phi \cdot (-f) & [\beta = \text{tyre effect}, \phi = \text{fuel effect}] \\
 &= \text{intercept} + \beta \cdot (a_0 + f) - \phi \cdot f \\
 &= (\text{intercept} + \beta \cdot a_0) + (\beta - \phi) \cdot f
 \end{aligned}$$

The stint's intercept **absorbs** $\beta \cdot a_0$, and f only ever appears multiplied by $(\beta - \phi)$. So from one stint, **only the difference $\beta - \phi$ is identifiable** — not β and ϕ separately. This is not a data-quality problem or a sample-size problem. It is exact algebra. Adding a million more laps does not help.

Result across 520 runs in 11 sessions:

Measure	Value
Runs exactly collinear	520 / 520 = 100%
Mean within-run correlation (age vs laps)	1.000
Sessions with singular Fisher information	11 / 11
Sessions unidentified without a prior	10 / 11
Cramér–Rao lower bound (best possible sd)	0.01607
Our model's reported sd	0.02215
Ratio (how close to optimal)	1.38×
Curvature share (the only data-driven escape)	6.0%

What each line means:

- *Singular Fisher information* means the likelihood surface has a perfectly flat direction. There is literally no unique maximum without extra information.
- The **Cramér–Rao bound** is the theoretical floor on how precise *any* unbiased estimator could be. Ours sits at **1.38×** that floor — near-optimal, not perfect, and we can say exactly how much room is left.
- **Curvature is the only escape.** The separation is exactly collinear only for the *linear* parts. A tyre degradation curve bends (warm-up early, cliff late); fuel burn-off is a straight line and never bends. That curvature is the sole data-driven channel through which the two can be told apart — and it carries just **6.0%** of the information. The other 94% must come from the physical prior.

Why this is the thing to lead with technically. Most entries will claim "our model separates fuel from tyre". We can state **exactly how much of that separation is data and how much is assumption**, prove the data part is 6%, and show our estimator is within 1.38× of the information-theoretic optimum. That is a fundamentally different level of claim, and it is the honest one.

PART 6 — EVERY CLAIM, AND WHETHER IT IS REAL

The team will be asked hard questions. Here is the exact strength of every claim we make, so nobody over-sells and gets caught.

6.1 The training/testing question — answer this one carefully

The question a judge will ask: *"What did you train on and test on? You only have lap times (y). You don't have the true wear rate (X , the label). So how can you say anyone's model fails?"*

The honest answer, in three parts:

(a) On synthetic data we have a genuine train/test setup. We generate a session, we know the true rate, we hide it, we estimate, we compare. That is a real held-out test against a real label. exp01 (0.0044 vs naive 0.0966) and exp05's degradation half (0.0041 vs naive 0.0748, 100% vs 0% coverage) are both of this kind. The limitation is honest and must be stated: **it is our own simulator**, so it proves mathematical correctness, not real-world accuracy.

(b) On real data we have no label, so we do not claim a supervised test. What we have instead are two things that do not need a label:

- **Consistency (exp03).** Estimate from practice, estimate from the race, see if they agree. Our error is 0.0807 against the naive method's 0.1471. **But the reference is our own model's race fit**, which shares our assumptions and therefore flatters us. Present this as a consistency result, never as ground-truth validation. The fix is scoped: score each method against its own race-derived estimate.
- **Falsification by physical impossibility (exp14).** This needs no label at all. Tyres cannot get faster as they wear. The naive method returns a negative rate in **74% of races and 53% of stints**. You do not need to know the true value to know that a negative value is wrong.

(c) So what can we legitimately say about the naive method? Exactly this:

"In 74% of real Grands Prix, the standard public method returns a physically impossible answer — it says the tyre got faster as it wore. That is checkable without any ground truth, because it is a sign error, not a magnitude error. On synthetic data where the true rate *is* known, the same method's 95% interval contained the truth in 0 out of 18 cases, and ours in 18 out of 18."

Everything in that paragraph is backed by a result file. Nothing in it claims we validated our magnitude against measured wear, because we did not.

6.2 Claims by strength

● Strong — directly proven, no caveat needed

Claim	Evidence
The naive method produces physically impossible answers in most real races	exp14: 74% of 77 races, 53% of 208 stints
On data with known truth, our estimator is essentially unbiased and the naive one is systematically biased low	exp01: bias +0.0012 vs -0.0966 over 25 seeds
Our 95% intervals really are 95% after conformal calibration	exp12: 94.7%; exp13: 95.2% over 69,206 laps
Fuel and tyre are exactly collinear within a stint	exp18: 520/520 runs, correlation 1.000, 11/11 singular
Our precision is within 1.38× of the theoretical optimum	exp18: CRB 0.0161 vs 0.0221
The method is not F1-specific	exp07: NASA C-MAPSS, RMSE 22.7, 100 engines, unmodified code
Degradation curves have four distinguishable shapes	exp17: 2,827 stints, BIC-selected
Our physics layer is internally correct	exp06: circuit direction from tyre load, 7/8

● Real but needs the caveat said out loud

Claim	The caveat
"45% better than naive at practice-to-race" (exp03)	The reference is our own model's race fit. It is a consistency test. Always say so.
"Cliff at ~72% through the stint, +0.20 s/lap" (exp17)	Real and well-measured, but fitting it does not improve forecasts except during warm-up (-26.5% overall).
"A driver effect exists" (exp15)	Only teammate-to-teammate, only 0.009 s/lap of spread, and it does not improve prediction (-0.88%, p = 0.066).
"Only 6% of the fuel/tyre separation is data-driven" (exp18)	This is a property of <i>our</i> parameterisation. A different model form would have a different split — though the exact collinearity is universal.
"We beat five alternatives" (exp05)	Only on degradation recovery . On lap-time CRPS we are third . Say both numbers.

● Things we tried, that failed, and now report as negatives

Idea	Outcome
Energy clock instead of lap clock	exp04: no meaningful difference (energy CV is only 2.3%)
True C1–C5 compound beats the HARD/MEDIUM/SOFT label	exp08: 6.4% worse , p = 0.096
Circuit geometry transfers between tracks	exp09: -3.3%, p = 0.22 — no detectable effect
Temperature or traffic explains our bias	exp10: temp ρ = -0.022, p = 0.835 — explains nothing
Matching stint depth fixes the bias	exp11: bias 6.9% worse , MAE 13.8% worse, p = 0.016
Driver style is a usable feature	exp15: prediction 0.88% worse

Thirteen refuted hypotheses is not a weakness in the write-up — it is the evidence that the surviving claims were actually tested.

6.3 Things we said and then had to take back

Listing these is deliberate. If the team knows what we retracted, nobody will repeat a dead claim in front of a judge.

1. **"Public FastF1-community tooling does lap-time-vs-age regression — exactly the method we show fails."** **✗ Retracted.** This appeared in an earlier summary. We have **no evidence** for it — no survey of community notebooks, nothing in the repo. The *method* (naive regression) is a fair strawman because it is the obvious first thing anyone does, and exp14 shows it fails. But we must **not** attribute it to a named community. Say "the standard naive approach", never "what the FastF1 community does".
 2. **"Circuit geometry is harmful."** **✗** Corrected to **"no detectable effect"** ($p = 0.22$). Only the *thermal* features are significantly harmful ($p = 0.0012$).
 3. **"The drift argument explains the bias."** **✗** Retracted twice, and replaced by the conformal interval, which fixes coverage without needing the mechanism.
 4. **"exp03 validates us against ground truth."** **✗** It validates practice-against-race **using our own model as the reference**. Consistency, not ground truth.
 5. **"Our PIT miscalibration is caused by sample size / by the fuel bug."** **✗** Wrong twice. The actual answer needed a three-region reading of the histogram; the model sits on the tails-vs-middle boundary and is leptokurtic.
 6. **"Per-driver rates differ by $1e-5$, so drivers don't matter."** **✗** That was a pinned optimiser parameter, not a fact about drivers. Rewritten.
-

PART 7 — HOW WE KEEP THIS FROM ROTTING

326 automated tests, 1 skipped, across 15 files:

File	Tests	What it protects
tests/unit/test_loader_discipline.py	63	Structural. Reads every experiment's source and fails if it calls <code>pd.read_parquet</code> directly, bypassing the fuel-counter repair. This is the test that would have caught our worst bug.
tests/integration/test_api_endpoints.py	28	Every dashboard route answers with strictly valid JSON — no bare <code>NaN</code> , which is not legal JSON and poisons a whole WebSocket frame.
tests/unit/test_conformal.py	27	Split conformal, including the finite-sample quantile correction.
tests/unit/test_adaptive_conformal.py	27	Online α updates, including the two fallback bugs from exp13.
tests/physics/test_thermal_wear.py	23	The physics layer.
tests/unit/test_curve.py	20	Broken-stick fitting and regime classification.
tests/physics/test_dynamics.py	18	Vehicle dynamics.
tests/unit/test_simulate.py	17	The synthetic generator itself.
tests/unit/test_circuit_features.py / test_compounds.py / test_live.py / test_rag.py	16 each	Reference data, compounds, live stream, retrieval.
tests/unit/test_corpus_selection.py	13	Session ordering — the bug where sorting by event <i>name</i> silently dropped Abu Dhabi, Australia and Austria.
tests/unit/test_fuel_counter.py	13	The fuel-counter fix, directly.
tests/unit/test_kalman.py	13	The filter and smoother.

Also: `scripts/check_results_fresh.py` verifies that no published document quotes a number older than the result file it came from.

PART 8 — HOW TO RUN / REBUILD ANY OF THIS

```
# Set up
python -m venv .venv
.venv/Scripts/pip install -r requirements.txt -r requirements-dev.txt

# Everything below works OFFLINE using data/demo/
.venv/Scripts/python -m pytest                # all 615 tests
.venv/Scripts/python -m uvicorn tyreminind.api.main:app --reload  # the dashboard API

# Rebuild the full four-season corpus (needs internet, takes hours, rate-limited)
.venv/Scripts/python scripts/build_corpus.py --years 2022 2023 2024 2025 --delay 6.0

# Rebuild the reference files
.venv/Scripts/python scripts/build_session_conditions.py --years 2025 --sessions R FP2 --delay 3.0
.venv/Scripts/python scripts/build_lineups.py --years 2025 --delay 2.0
.venv/Scripts/python scripts/build_circuit_features.py --year 2025 --geometry-only --delay 1.5

# Re-run every experiment, in dependency order
bash scripts/rerun_all.sh
```

Dependency order matters and getting it wrong gives you numbers that quietly disagree rather than an error: exp09 and exp10 read exp08's estimates; exp10 and exp12 read exp03's comparisons. `rerun_all.sh` encodes the correct order.

PART 9 — THE EXPERIMENTS WE WILL RUN NEXT

Eighteen experiments are done. These are the next fourteen, in priority order. Each one has: the question, what data it needs, how we'll do it, and — most importantly — **what counts as a pass and what counts as a fail**, decided in advance so we can't move the goalposts afterwards.

exp19 — Remove the circularity from the practice-to-race test ● highest priority

The problem it fixes. Today exp03 compares our practice estimate to our own model's race fit. That shares our assumptions, so it flatters us. It is the single weakest point in our evidence and a sharp judge will find it.

The fix. Score every method against **its own** race-derived estimate. The naive method gets judged on whether *the naive method* agrees with itself from Friday to Sunday. We get judged the same way. Nobody gets a home-ground reference.

Data: already have it. 42 events, 94 comparisons. **Cost:** about 25 minutes of compute. **Pass:** we stay ahead on self-consistency. **Fail:** the gap shrinks a lot — in which case we say so and lead with exp14 instead, which needs no reference at all.

exp20 — Validate against a tyre model we did not write ●

The problem it fixes. Our ground truth today is our own simulator. "You graded your own homework" is a fair criticism and we should remove it.

The idea. The official EA F1 game publishes a UDP telemetry stream that includes a **true per-wheel tyre wear percentage**, plus tyre temperatures, pressures and fuel mass. It's a commercial tyre model built by people with real Pirelli relationships — and critically, **we didn't write it**.

Method. Drive or auto-run full race stints. Record the wear channel secretly. Feed our model **only** what the public feed would give — lap time, compound, tyre age. Compare our estimate to the hidden truth.

Pass: our error against an independent truth is in the same range as exp01's 0.0044, and the naive method still fails.

Fail: our error blows up, which would mean our simulator was too kind — and we'd need to know that.

Why it matters for the demo too: it gives us a truth engine for the two-car 3D comparison that isn't our own model, so the demo can't be accused of being rigged.

exp21 — Reproduce a single-circuit calibration and test whether it travels ●

The question. The existing TrackShift engine is a five-parameter formula calibrated at Bahrain. Does a single-circuit calibration generalise?

Method. Build that five-parameter formula ourselves, honestly and in good faith, fitted at one circuit. Then score it **leave-one-circuit-out** across our 26 circuits, next to our model, on identical data.

Why do this. It's the most judge-legible chart we can make: one line per method, 26 circuits, same axis. And it uses their method as the baseline rather than a strawman we invented, which is both fairer and more convincing.

Prior evidence it will struggle: exp09 already shows circuit transfer fails — geometry gives -3.3% ($p = 0.22$), temperature actively hurts ($p = 0.0012$).

Pass: we quantify the transfer gap with a p-value. **Fail:** the single-circuit formula travels fine — which would be a genuinely interesting and publishable negative result about our own assumption.

exp22 — Build and test a grip index

The question. Can we produce a meaningful "grip level" from public data?

The honesty constraint, stated up front. A true traction coefficient needs measured tyre force and slip, from load cells and wheel-speed sensors that are team-private. **We will not print a fake number.** We'll publish a normalised grip index from 0 to 1 — achieved lateral acceleration divided by the expected maximum for that circuit and conditions.

Data needed: FastF1 car telemetry (speed, throttle, brake, gear at ~ 4 Hz) plus position data, for our 203 sessions. Overnight download, 2–6 GB.

The test that makes it real: does the grip index predict the *next* lap's time loss better than tyre age alone? If it doesn't, it's decoration and we say so.

exp23 — Puncture risk: the experiment we ran, and the answer was no RESOLVED

This experiment is already finished, and it did not go the way we planned. We collected the labels. There are not enough. This section records what we found because it changes what we ship — and because it applies to everybody, not just us.

What we did. `scripts/build_failure_labels.py` pulls every race classification from the public results table and groups the retirement causes by physical origin. We scanned the whole Pirelli era, **2011–2022, 5,057 result rows**.

What came back:

Cause group	Events	What it contains
Competing risks	351	Collision 190, Accident 102, Collision damage 45, Spun off 13, Debris 1
Vehicle (non-tyre)	151	Gearbox 54, Brakes 50, Suspension 37, Transmission 8, Driveshaft 2
Wheel assembly	24	Wheel 16, Wheel nut 8
Tyre proper	17	Puncture 12, Tyre 5

Seventeen tyre failures in twelve seasons.

Three separate limits, all confirmed by measurement:

- The positive class is tiny.** 17 events across twelve seasons of changing regulations, tyre constructions and cars. `wheel nut` is a *pit-stop* failure, not a tyre one, so it does not help.
- Detailed causes stop after 2022.** From 2023 the public results carry only `Finished` / `Lapped` / `Retired`. The modern era has no cause labels at all.

3. **Race control messages do not contain punctures.** We checked three 2024 races: **0 tyre-related messages out of 249.** They carry flags, DRS zones and pit-lane status. An earlier version of this document claimed these messages would multiply the positive class. **That was wrong and is retracted.**

The conclusion, stated plainly:

A validated puncture *probability* cannot be built from public data. Not by us, and not by anyone else either.

So what do we ship for metric 5? A **structural exposure index** — accumulated energy, thermal stress, laps beyond expected life, kerb exposure — which is monotone in risk, physically motivated, and **explicitly labelled as an index, not a probability.** The UI must never print "7.2% puncture risk", because that number would be invented.

Why this is a strong result rather than a failure. It is the sharpest, most respectful question we can ask about any competing system, including the existing TrackShift engine which lists "threshold-based and probabilistic puncture risk scoring":

"We went looking for the labels needed to calibrate a puncture probability. There are seventeen in twelve seasons. So we report exposure rather than probability — and we would genuinely like to know what a probability here is calibrated against."

And the unexpected payoff. The **vehicle** group has **151 events** — roughly nine times the tyre count — across five component families with workable per-family counts (Gearbox 54, Brakes 50, Suspension 37).

The "full vehicle health monitoring" item that the current system lists as *future work* is better supported by public data than the puncture metric it already ships. That is where the hazard model should go, and that is exp30.

Artefact: `data/reference/failure_events.json`.

exp24 — Calibrated remaining useful life, in laps ●

The question. "How many laps of useful life are left?" with an interval, not a single number.

What we already have. We do exactly this on jet engines (exp07: 22.7 cycles RMSE, all 100 engines). We have the F1 plumbing too — `/health-timeline` and `/pit-window` endpoints already exist.

What's missing. Two things. First, a calibrated interval — port the adaptive conformal layer from exp13 so the output is "7 laps left, 95% interval [4, 11]". Second, a definition of when useful life ends — and we already have a data-derived one rather than a hand-set threshold: **the cliff onset from exp17**, measured at 71.9% through a stint, per compound (HARD lap 22.5, MEDIUM 14.0, SOFT 16.0).

Pass: coverage within 2 points of nominal on held-out stints.

exp25 — Redo the energy and circuit experiments with real telemetry ●

exp04 (energy clock) ran on 4 stints with proxy energy. exp09 (circuit transfer) ran with `mean_abs_lateral_g` set to `None` because we never downloaded telemetry.

Both deserve a rematch with measured per-corner loads and real lateral g across all 203 sessions. It is entirely possible that exp04's "no meaningful difference" flips once energy is measured rather than approximated — and if it does, we report the reversal openly.

exp26 — Replace our invented traffic index with a measured one ●

We *derive* traffic from differences in lap start times. OpenF1 publishes **measured** `gap_to_leader` and `interval` every ~4 seconds, free for 2023 onwards.

The experiment: swap the derived index for the measured one and see whether any result changes. If nothing changes, that's a strong validation of a clever shortcut. If things change, we learn our proxy was hiding something.

exp27 — Compound allocations for all four seasons ●

Today we have Pirelli's C1–C5 nominations for 2024 only, assembled by hand. There is an open-source parser that reads the official FIA PDFs, including tyre compounds. Scripting it gives us 2022, 2023 and 2025 without manual transcription.

Then re-run exp08 on four seasons instead of one. Current answer: the true compound is 6% *worse* than the weekend label. Four seasons will tell us whether that holds.

exp28 — Does a better estimate actually win races? ● the business experiment

The question nobody has asked us yet, and will. Everything so far measures *accuracy*. This measures *value*.

Method. Replay every real race. At each pit decision point, compute what the naive method would have recommended and what we would have recommended. Score both against what actually happened, in **seconds lost and positions lost**. Our `/regret` endpoint already does this arithmetic for a single call.

The output is one sentence for the pitch: "Across N races, using the naive estimate instead of ours would have cost an average of X seconds and Y positions per race." That converts a statistics result into a number a team principal cares about.

exp29 — Multi-compound strategy simulation under uncertainty ●

Today the strategy simulator takes point estimates and returns a point answer: "one-stop is 3.2 s faster". That's not actionable, because it hides the risk.

Method. Monte Carlo over the *conformal* degradation distribution for each compound, producing a distribution over outcomes.

Output: "One-stop wins 61% of the time, two-stop 39%, and the crossover is at a track temperature of about 42 °C." A strategist can act on that.

exp30 — A second cross-domain proof 🟡

exp07 proved the engine works on turbfans. One dataset could be luck. A second domain — bearings, batteries or industrial pumps — turns "it worked once" into "it's general".

This is also the direct evidence for the "full vehicle health monitoring" ambition. And we already have adjacent labels in the F1 data itself: **Suspension (37), Brakes (50) and Gearbox (54)** retirement causes, which the same hazard machinery from exp23 handles with no new method.

exp31 — Partial pooling across events 🟢

Right now each event is fitted fairly independently. A hierarchical model that shares strength across events should tighten estimates at events with little practice running — which is exactly where exp10 found our bias is worst (shorter practice stints → more bias).

exp32 — Wet-weather regime 🟢

We currently **exclude** all wet running and say so honestly. Wet degradation is a genuinely different physical process — driven by water clearance and overheating on a drying line, not by rubber abrasion. Modelling it is a separate model, not a tweak. Worth doing, clearly out of scope for now.

exp33 — Live-feed integration test 🟢

We have the live architecture. The remaining step is pointing it at a real feed rather than a replay, and confirming the adaptive interval still lands on 95% when data arrives late, out of order, or not at all.

exp34 — Driver-in-the-loop response 🟢

If a driver lifts and coasts, energy through the tyre drops, and the remaining life should extend. We have the physics (`frictional_power_proxy` , `lap_energy`). The experiment is whether our predicted life actually responds correctly to a real change in driving style — a closed-loop test rather than an open-loop one.

Priority summary

Priority	Experiments	Theme
✅ Done	exp23	Answered: the labels do not exist. See above.
🔴 Do first	exp19, exp20, exp21, exp24, exp28, exp30	Fix our weakest evidence, build the missing metrics, prove business value
🟡 Do next	exp22, exp25, exp26, exp27, exp29, exp30	Better features, better data, wider proof
🟢 Later	exp31, exp32, exp33, exp34	Refinement and productionisation

PART 10 — BUILDING ON THE EXISTING TRACKSHIFT WORK

The previous team built a strong system. Our design principle is deliberate:

Keep their five metrics, their 18 alerts and their 17-step action ladder as the visible interface. Replace what computes them. Add the layer they cannot have.

Three reasons, and none of them is politeness:

- 1. **Their metric set is genuinely right.** Grip, tread, degradation, energy, puncture risk is exactly what a race engineer wants on a pit wall.
- 2. **The 18 rules are institutional knowledge.** Rules encode "we have seen this go wrong before". Deleting them throws that away. Wrapping them keeps it as a safety floor and as the explanation layer.
- 3. **Continuity reads as engineering.** "We replaced everything" sounds dismissive. "We kept the interface and made every number honest" sounds like a team that understands the product.

10.1 Metric by metric — where we are, honestly

Their metric	Do we have it today?	What we'll do
Degradation Rate 	✔ Yes, and stronger	Already validated 4 ways across 84 event-seasons with calibrated intervals. Add exp21 as the head-to-head benchmark.
Tyre Energy ⚡ (laps of life left)	🟡 Mostly — we do this on jet engines and have the F1 plumbing	exp24: add a calibrated interval and use the measured cliff as the threshold.
Grip Level 🛖	✖ No — but we have the physics layer	exp22: build a normalised 0–1 grip index from telemetry. Never a fake traction coefficient.
Tread Remaining 	✖ No units — our wear model outputs arbitrary units	Report % of usable life , not millimetres. Anchor and validate via exp20. See 10.2 — read it before anyone asks.
Puncture Risk ⚠	✖ Nothing	exp23 is done and negative : only 17 tyre failures exist in 12 seasons of public data, so no probability can be calibrated. We ship a structural exposure index , labelled as an index and never as a percentage.

10.2 The tread-depth question — decide this now, not in the room

We cannot honestly print millimetres. There is no public measurement of F1 tread depth to anchor against — we searched thoroughly and it does not exist.

Option	What it means	Verdict
% of usable life remaining	"62% left, 95% interval [48%, 74%]"	✅ Ship this. Unit-free, honest, and it's what a strategist acts on anyway.
Anchor to an independent tyre model	Calibrate our arbitrary units against the game's published wear %	✅ Do this for validation (exp20).
Claim millimetres	"2.1 mm remaining"	❌ Never. One informed question — "measured against what?" — and the whole deck loses credibility.

Say it out loud in the meeting: *"They report remaining depth. We report remaining usable life, because depth is not measurable from public data and we would rather be right than precise."* That sentence earns more than a fabricated millimetre ever will.

10.3 The decision layer — what replaces 18 rules and a 17-step ladder

Their logic is deterministic: metric crosses threshold → alert fires → ladder picks an action. Ours is decision-theoretic, and we already have the pieces (/pit-window , /strategy , /regret).

THEIR ARCHITECTURE		OURS	
metric →	a single number	→	a calibrated probability distribution
logic →	18 fixed thresholds	→	choose the action with lowest expected cost
output →	an alert + a ladder step	→	action + expected gain + interval + regret if wrong
audit →	(none)	→	their 18 rules still run, as a safety floor

Three concrete changes:

- 1. Pit calls become an expected-cost calculation.** Pitting one lap early costs a known ~20 seconds. Pitting one lap late costs an unknown amount that depends on the *distribution* of degradation. We minimise expected cost over the calibrated interval, not over a point estimate. That's a direct, numerical answer to "why is your recommendation better".
- 2. Keep all 18 rules as a veto layer.** If our model says "stay out" and rule 14 says "puncture risk elevated", **the rule wins**, and the screen shows both and why they disagree. Disagreement between a learned model and encoded expertise is the single most useful thing you can show a race engineer.
- 3. Make every decision auditable after the race.** Replay each decision point: what we'd have said, what was done, what it cost. This is the **"post-race validation tools"** clause that is literally in the problem statement — and it is not mentioned anywhere in the current system.

PART 11 — THEIR FOUR "FUTURE OPPORTUNITIES", DESIGNED

11.1 Integration with live race telemetry streams

Status: substantially built already. We have a live streaming endpoint, a Kalman filter updating lap by lap, and adaptive conformal intervals that **self-correct during the session** — 95.2% coverage over 69,206 laps, 0.2 points off target. The live frame already reports its own running coverage, so it can be audited *while it runs*.

Remaining work: swap the replay source for a real live feed. The consumer side doesn't change at all.

Why it's differentiated: a live system whose error bar corrects itself in flight and publishes its own accuracy is a different class of product from one that streams point estimates.

11.2 Driver-in-the-loop strategy recommendation

Design. Driver input changes energy through the tyre, which changes the wear rate, which changes the remaining life and therefore the pit window. We already have `frictional_power_proxy()` and `lap_energy()`. Wiring driver behaviour → energy → revised remaining life is a real closed loop, tested by exp34.

This is also where the 3D demo lives — with one structural warning.

The two cars do **not** differ in how they drive. They differ in **when they pit**. Our model doesn't make a car faster; it produces a better estimate, which produces a better pit window, which produces a better finishing position.

So the honest demo is: same car, same pace physics, two strategies — one planned off the naive estimate, one off ours — and the gap at the flag is the result.

And the truth engine must not be ours. If the simulated race is driven by our own model, the demo is rigged by construction and a technical judge will see it immediately. The car without our model has to lose *because the underlying tyre truth punished its bad estimate* — and that truth must come from somewhere we didn't write. That is the second reason exp20 matters.

11.3 Multi-compound strategy simulation

Covered by exp29. The upgrade is propagating uncertainty through the simulation, so the output is a distribution over outcomes rather than a single predicted time.

11.4 Expansion to full vehicle health monitoring

This is the one we are furthest ahead on — and they listed it as future work.

- exp07 already proves the estimator is domain-agnostic: unmodified code, NASA turbofan engines, 22.7 cycles RMSE.
- A cross-industry endpoint already exists in our API.

- And the labels for the rest of the car are already in hand: **Suspension (37), Brakes (50), Gearbox (54)** — 151 vehicle events against only 17 tyre events, so the hazard model belongs here rather than on punctures.

The line to say: *"They listed full vehicle health as a future opportunity. We shipped the cross-domain proof and validated it on aircraft engines."*

PART 12 — THE PLAN, IN ORDER

12.1 Data we still need to pull

#	Data	What it unlocks	Cost
1	Telemetry for our 203 existing sessions	Grip Level, real tyre energy, real traffic, per-corner loads	Overnight, 2–6 GB
2	Failure labels	Done. 543 events, 2011-2022. Tyre labels too few (17); vehicle labels usable (151).	complete
3	Independent tyre-model capture harness	External validation (exp20) + honest truth engine for the demo	1–2 days
4	FIA PDF compound parser	Compound allocations for 2022/23/25 (exp27)	~half a day
5	OpenF1 stints / intervals / pit	Measured traffic replacing our derived index (exp26)	~half a day
6	Track geometry (GeoJSON, centre lines)	The 3D demo	Small
7	2018–2021 seasons	More laps of the same eight columns	Lowest value — probably skip

Say this if asked "don't you need more data?": *"Not really — 92,000 laps is already a lot for this question, and exp18 proved the limit is algebra, not sample size. What we're short of is **labels** and **extra columns**, not more rows. Items 1, 2 and 3 fix that. Item 7 is the one that feels like more data and is the least useful."*

12.2 Build order

Phase	Work	Why in this position
1	Failure labels → hazard model → calibrated Puncture Risk	Cheapest real win. Fills our only completely empty metric.
2	Telemetry download → Grip Level + real Tyre Energy	Unblocks two metrics. Runs unattended overnight.
3	Calibrated remaining life using the measured cliff	Turns machinery we already have into their metric #4.
4	exp19 + exp21 — de-circularise, then the head-to-head benchmark	Fixes our weakest evidence and produces the clearest chart.
5	Decision layer — expected-cost pit calls, 18 rules as safety floor, post-race audit	Replaces the engine while keeping their interface.
6	exp20 — independent truth engine	Also the honest foundation for the demo.
7	exp28 — the value experiment (seconds and positions)	The one number a team principal cares about.
8	3D two-strategy demo on the independent truth engine	Presentation layer, built last, on solid ground.

12.3 The one thing we should ask the mentors for

What was in the telemetry dataset the previous team used?

This genuinely changes the plan:

- **If it contained per-lap tread depth or wear %**, then they had labels we don't. We should request the same dataset — it would immediately solve the tread-depth units problem and strengthen every validation we have.
- **If it was lap times and car channels with no wear label**, then their "Tread Remaining" is a modelled quantity just like ours, and the honest-units argument in 10.2 applies to their system too.

Ask it plainly. Either answer helps us, and asking shows we've read their work carefully rather than just talked over it.

12.4 One-line summary of our whole position

"Everyone can build a dashboard that shows a degradation number. We built one that is right, that tells you how confident it is and is actually right that often, that proves the standard method is broken in 74% of real races, and that states mathematically how much of the answer is data and how much is assumption. And then we ran the same code on jet engines to prove it isn't an F1 trick."

PART 12B — THE STEP-BY-STEP BUILD PLAN (4 PEOPLE)

Everything above says *what* to build. This part says *how*, step by step, with the exact files, the exact order, and what "done" means for each step so nobody has to guess.

Team shape assumed: 4 members. The four workstreams below are deliberately chosen so they **do not touch the same files**, which means four people can work in parallel on four branches with almost no merge conflicts.

Stream	Owner	Mostly touches	Depends on
A — Data & Labels	Member 1	<code>scripts/</code> , <code>data/</code>	nothing (start immediately)
B — New Metrics	Member 2	<code>src/tyremind/models/</code> , <code>src/tyremind/physics/</code>	A1, A2
C — Decision Layer & API	Member 3	<code>src/tyremind/api/</code> , <code>apps/web/</code>	B (loosely)
D — Validation & Demo	Member 4	<code>experiments/</code> , <code>docs/</code> , the 3D sim	A3

Branch rule: everyone branches off `main` , nobody commits to `main` , one branch per stream (`nitya/dev` , `<name>/data` , `<name>/metrics` , `<name>/demo`).

STREAM A — DATA & LABELS

A1. Download telemetry for all 203 sessions

Why: unlocks Grip Level and real tyre energy. This is the single biggest unlock and it runs while you sleep.

Steps:

- Create `scripts/build_telemetry.py` , modelled on the existing `scripts/build_corpus.py` (copy its rate-limiting and retry structure — do not reinvent them).
- For each session already in `data/season/` , pull `session.car_data` and `session.pos_data` from the cache.
- Reduce per lap — **do not store raw 4 Hz samples for 203 sessions, that is gigabytes of noise**. Store per lap: mean and peak lateral g, mean and peak longitudinal g, total frictional energy, time at full throttle, time on the brakes, and per-corner load shares. `physics/dynamics.py` already computes every one of these; you are calling existing functions, not writing new physics.
- Write to `data/telemetry/<session_id>.parquet` , one row per driver per lap, with `driver` and `session_lap` as the join keys to the existing lap table.
- Add a `.quality.json` next to each, in the same shape as the lap-table ones, so the telemetry is auditable the same way.
- Run with `--delay 6.0` overnight. Add a second retry pass, as `build_corpus.py` already does.

Done when: every session in `data/season/` has a matching telemetry file, and a join on `(driver, session_lap)` loses less than 2% of laps. **Report the loss rate — don't hide it.**

Trap to avoid: telemetry lap boundaries and timing lap boundaries do not always agree exactly. Join on driver + lap number, never on timestamps.

A2. Pull failure labels

Why: this is what makes Puncture Risk real instead of decorative, and it is only about half a day's work.

Steps:

1. Create `scripts/build_failure_labels.py`.
2. Pull retirement statuses from the public results API for every race in our corpus. The relevant status codes are `Puncture`, `Tyre`, `Wheel`, `Wheel nut`, `Wheel bearing` — plus `Collision damage` which you must keep **separately** as a competing risk, and `Suspension` / `Brakes` for the vehicle-health extension.
3. Pull race control messages from the session data — these catch punctures that happened **during** a race without causing a retirement. This roughly multiplies the positive class and is the difference between a usable dataset and one that's too small.
4. Write `data/reference/failure_events.json`: one row per event with `year`, `round`, `driver`, `lap`, `cause`, `source` (results table or race control), and `confidence`.
5. **Write a validation script.** Cross-check a sample of events against the race classification. Failure labels scraped from two sources will disagree sometimes; record which source won and why.

Done when: we have a labelled event table and a printed count of positives per season. **Be honest in the write-up about how few there are.**

A3. Independent tyre-model capture harness

Why: removes the "you graded your own homework" criticism, and gives the 3D demo a truth engine we didn't author.

Steps:

1. Create `scripts/capture_external_truth.py`.
2. Open a UDP socket on the documented port. Parse the packet header, then the packets carrying lap data, car status and car damage.
3. Extract per lap: **true tyre wear % per wheel**, tyre surface and carcass temperature, tyre pressure, fuel in tank, lap time, compound, tyre age.
4. Write two files per session — a **public** one containing only what the real F1 timing feed would give (lap time, compound, tyre age, session lap), and a **truth** one containing the wear channel.
5. **The truth file must never be read by any estimator.** Enforce it the same way we enforced the loader rule: add a structural test to `tests/unit/test_loader_discipline.py` that fails the build if any file in `src/` or `experiments/` imports or opens the truth path. We already have this pattern working — copy it.
6. Collect at least 20 full race stints across at least 3 compounds and 3 circuits.

Done when: exp20 can run. **Watch out for:** the wear channel is a percentage, not millimetres, and it is a *model's* wear, not a measured one — say both of those in the write-up.

A4. Compound allocations for all four seasons

1. Create `scripts/build_compound_allocation.py` using the open FIA PDF parser.
2. Extract the nominated C1–C5 set per event for 2022, 2023 and 2025.

3. Merge with our hand-built 2024 file, **keeping the existing** `confidence` **and** `source` **fields** so hand-entered and parsed rows stay distinguishable.
4. Re-run exp08 on four seasons.

Done when: `compound_allocation.json` covers ~84 events instead of 24.

A5. Measured traffic from OpenF1

1. Create `scripts/build_measured_traffic.py`.
 2. Pull the intervals endpoint for every race from 2023 onwards (free tier).
 3. Align the ~4-second interval samples to laps, producing a measured gap-to-car-ahead per driver per lap.
 4. Add it as a **new column** — `traffic_measured` — alongside our derived `traffic_index`. **Do not overwrite the derived one.** The comparison between them is exp26, and you destroy the experiment if you replace the column.
-

STREAM B — THE NEW METRICS

B1. Grip Level (needs A1)

1. New module `src/tyremind/physics/grip.py`.
2. Compute achieved lateral acceleration per corner from telemetry (already available via `dynamics.lateral_acceleration`).
3. Estimate the expected maximum for that circuit and conditions — fit the upper envelope of achieved lateral g across all drivers, early in the session, on fresh tyres.
4. Grip index = achieved ÷ expected, clipped to 0–1.
5. **Then test it before shipping it:** does adding the grip index improve prediction of the *next* lap's time loss, over tyre age alone? Use the same leave-one-out structure as exp09, and the same Wilcoxon test. **If it does not improve prediction, say so and ship it as a diagnostic display only, not as a model input.**

Non-negotiable: the UI must label it "grip index (0–1)", never "traction coefficient", and never print a μ value.

B2. Tread Remaining, as % of usable life

1. Extend `src/tyremind/physics/wear.py` with a function that converts cumulative `wear_increment` into a fraction of usable life.
 2. Define "usable life" from the **measured** cliff onset in exp17 — per compound, with the median and the IQR — not from a hand-set constant.
 3. Attach a conformal interval, reusing `models/conformal.py`.
 4. Output shape: `{"life_remaining_pct": 62.0, "interval": [48.0, 74.0], "basis": "cliff onset, exp17, HARD, n=180"}`. **Always carry the basis string** so the number can never be quoted without its source.
 5. Validate against the independent truth from A3 in exp20.
-

B3. Puncture Risk (needs A2)

1. New module `src/tyremind/models/hazard.py`.
2. Build the risk set: one row per driver per lap, with an indicator for whether a failure occurred on that lap, and a censoring flag for stints that ended in a normal pit stop.
3. Fit a discrete-time hazard model — a logistic regression on the hazard is the right starting point, and it is deliberately simple because the positive class is tiny. Features: tyre age, cumulative energy, thermal stress, compound, circuit kerb exposure.
4. **Handle competing risks properly.** Debris punctures and wear-driven structural failures are different events. Fit them as separate causes; do not pool.
5. **Partial pooling across circuits**, because some circuits will have zero events and a per-circuit model would confidently predict zero risk there.
6. Calibrate: reliability diagram, and a Brier score decomposition.
7. **Publish the calibration curve next to the number, always.** With this few positives, the calibration curve *is* the result.

The honest framing to build into the UI: *"elevated risk, 2–9%, based on 41 historical events"* beats *"7.2% risk"* every time.

B4. Calibrated Remaining Useful Life

1. Extend the existing health-timeline logic to emit laps-remaining rather than a health score.
 2. Wrap it in the adaptive conformal layer from `models/conformal.py` — the same one that achieved 95.2% over 69,206 laps.
 3. Validate coverage on held-out stints; target within 2 points of nominal.
 4. Reuse the exp07 evaluation harness so the F1 RUL and the turbofan RUL are **scored by the same code**. That shared harness is itself a selling point.
-

STREAM C — THE DECISION LAYER

C1. Expected-cost pit decisions

1. New module `src/tyremind/models/decision.py`.
2. Define the loss function explicitly and document it: pitting early costs a known pit-lane delta; pitting late costs the integral of degradation over the laps you stayed out, which is **a distribution, not a number**.
3. Minimise expected loss over the conformal predictive distribution, sweeping candidate pit laps.
4. Return the recommended lap, the expected gain over the next-best option, the interval, and the regret if the recommendation is wrong.

Acceptance test: when the degradation distribution is wide, the recommendation must become *more conservative*, not identical to the point-estimate answer. If widening the interval doesn't change the recommendation, the loss function is wrong — this is the bug to look for.

C2. Rules as a safety layer

1. New module `src/tyremind/models/rules.py` — encode the 18-rule catalogue with its supersede logic, faithfully.
2. Run rules **alongside** the probabilistic engine, never inside it.
3. On disagreement: **the rule wins**, and the response carries both verdicts plus a plain-language explanation of the conflict.
4. New endpoint `/api/session/{id}/decision` returning: model recommendation, rule verdict, agreement flag, and the explanation.

This disagreement view is a feature, not a fallback. It is the single most useful screen for a race engineer, and it is what "building on their work" looks like in code rather than in a slide.

C3. Post-race validation tools

The problem statement asks for these explicitly and the current system does not mention them.

1. New endpoint `/api/session/{id}/audit`.
2. Replay every decision point in a completed race: what we would have recommended, what was actually done, and the cost of the difference in seconds.
3. Aggregate to a per-race scorecard.
4. Feed exp28 from this — it is the same computation.

C4. Dashboard

1. Five metric cards matching their layout exactly: Grip, Tread, Degradation, Energy, Puncture Risk.
2. **Every card shows an interval, not just a number.** This is the visual signature of the whole project — if a judge takes one screenshot away, it should be this.
3. Alert panel showing model and rules side by side, with disagreements highlighted.
4. Post-race audit view.

STREAM D — VALIDATION & THE DEMO

D1. Run exp19 immediately

This is cheap, it is already scoped, and it removes our weakest claim. **Do it in week one** so that if the result is unfavourable we have plenty of time to change the story.

D2. exp21, the head-to-head benchmark

1. Implement the five-parameter single-circuit formula honestly, in good faith. Do not build a strawman — if it beats us somewhere, that is a finding.
2. Score leave-one-circuit-out across 26 circuits.
3. One chart, both methods, 26 circuits, same axis.

D3. exp28, the value experiment

Read positions and seconds out of the audit tool from C3. One sentence for the pitch: *"using the naive estimate would have cost X seconds and Y positions per race."*

D4. The 3D demo

1. Track geometry from the open GeoJSON circuit files and the open racetrack centre-line database.
2. Render with a standard browser 3D library — **keep it simple: two cars, a track line, a lap counter and a live gap**. Nobody is scoring the graphics.
3. The race is driven by the **independent** truth engine from A3.
4. Car 1 pits on the naive estimate. Car 2 pits on ours. Same pace model, same everything else.
5. Overlay both estimates against the hidden truth, updating live, so the audience can see the naive estimate drifting away from reality before the pit call goes wrong.

The rule that protects this demo: the truth engine must not be our model. Write it in the code comments so nobody "simplifies" it later. If our model generates the truth *and* makes the prediction, the demo proves nothing and a technical judge will spot it in thirty seconds.

Suggested sequencing

Phase	Stream A	Stream B	Stream C	Stream D
1	A1 telemetry download (overnight), A2 failure labels	<i>(read the code, write tests)</i>	C1 decision module	exp19
2	A3 capture harness	B3 puncture hazard, B4 RUL	C2 rules layer	exp21
3	A4 compounds, A5 traffic	B1 grip, B2 tread %	C3 audit endpoint	exp20, exp28
4	<i>(re-runs)</i>	validation & calibration	C4 dashboard	3D demo, exp25–27
5	—	—	polish	rehearse, re-render this document

Two rules that will save the whole project

1. **Re-run** `scripts/rerun_all.sh` **after any data change, and respect the dependency order.** exp09 and exp10 read exp08's output; exp10 and exp12 read exp03's. Get the order wrong and the experiments quietly *disagree with each other* instead of failing loudly. We have been bitten by this exactly once and it cost a day.
2. **Never bypass the loaders.** Read lap tables through `read_lap_table`, never through `pd.read_parquet` directly. A structural test enforces this — do not delete it to make a build pass. That test exists because a bug we had already fixed silently came back in five files and produced pre-fix numbers that looked completely plausible.

PART 13 — HOW TO DOWNLOAD AND SHARE THIS DOCUMENT

This file is at:

```
D:\TrackShift Innovation Challenge\docs\data_doc.md
```

It is **not committed and not pushed** — `git status` will show it as an untracked file on branch `nitya/dev`, and it will stay that way until someone deliberately adds it.

Pick whichever is easiest:

- 1. Just open it.** In VS Code, `Ctrl+Shift+V` on the file gives a rendered preview. Everything here is plain Markdown.
- 2. Copy it out of the folder.** Open `D:\TrackShift Innovation Challenge\docs\` in Explorer and copy `data_doc.md` anywhere — Desktop, a USB stick, a Drive folder.
- 3. Send it to the team as-is.** Markdown renders natively in WhatsApp Docs, Slack, Discord, Notion, Google Docs (paste), and GitHub. No conversion needed.
- 4. Make a PDF.** The repo already has a renderer:

```
.venv/Scripts/python scripts/render_pdfs.py
```

Or, without any tooling: open the file in VS Code → preview → right-click → Print → Save as PDF.

- 5. If you want it on your phone or another machine,** the simplest route is to copy the file into any cloud-synced folder. Do **not** `git push` it — the whole point is that it stays off the shared repo.

Reminder for the team: this document contains our honest internal assessment, including the six failed hypotheses and the six retracted claims. That is exactly what makes it useful for preparing for judge questions — and exactly why it is not in the public repo.